

Distributed and Cloud Computing Assignment 2

Haibin Lai
12211612

1. Introduction

In this assignment, we need to implement a simple Merch Store backend with 3 micro service: API service, DB service and Logging Service. For API Service, we use An OpenAPI specification YAML to generate a Python **Flask** RESTful API Service. And we use gRPC to communicate with DB and Logging Service, which are generated by Proto files. All of them run in Docker network, and we support 2 RESTful API Service using **NGINX**.

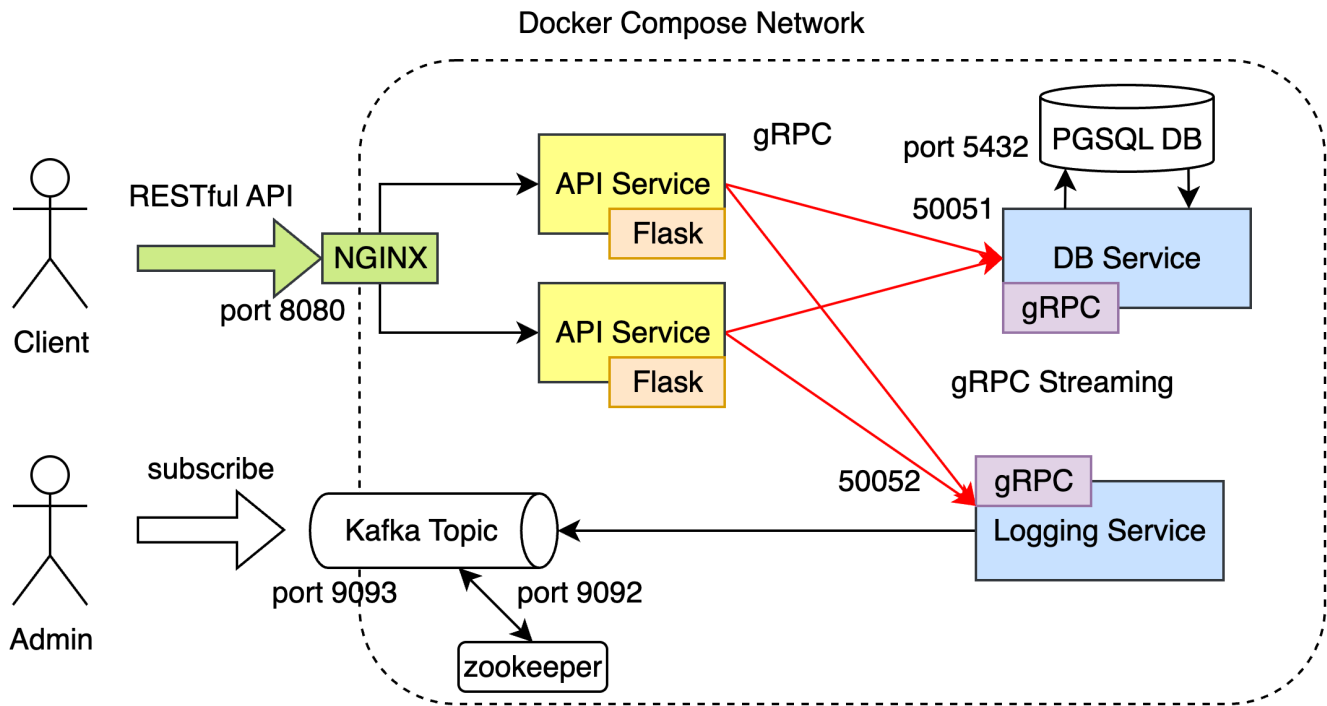
The rest of the report are organized as following:

- For section "Overall Architecture", we introduce our main system design and philosophy.
- For section "Setup", we discuss how we mainly build the system. And we also discuss our procedures of our implementation for each component, like API decision, Database CRUD design, gRPC code implementation and docker network design.
- For section "API Design", we discuss APIs authentication and authentication logic.
- For section "Field Data", we discuss how we select data types for different definitions.
- Section "gRPC verification" selects an arbitrary Proto message from your definition and analyze how it is encoded into binary format.
- Section "client-side streaming gRPC" explain how the client-side streaming RPC works.
- Section "Docker and Docker Compose" shows docker files that let each service communicate with each other.
- Section "Experiments" shows swagger UI to show our API, and how we observe log from kafka. We also use tool `hey` to show our system's QPS.

The report is a little long, since there are many details in the code.

2. Overall Architecture

All of the service and database, Kafka and zookeeper run in Docker. For clients, they can use port `8080` to send RESTful API. The NGINX Server will catch the requests and send them to API Service with load balance scheduling (like Round Robin). The API Service runs in Flask framework will receive and deal with the requests. They will send gRPC communication to DB Service and Logging Service to deal with the requests. The DB Service will communicate with PG database on port `5432`, while Logging Service will put logging message from port `9092` to Kafka, which the Administrator and know the logging information.



- The API Service include every API described in assignment.
- DB Service provide CRUD for products, users and orders.
- Logging Service provide communication with Kafka.
- RPC proto files, OpenAPI YAML file and Docker Compose File are provided.
- NGINX and it's config is also listed in file.

We also discuss some detail problems in these design:

1. OpenAPI type of accessing API
2. Generate JWT token when login, and use JWT to check accounts and orders
3. Use string instead of float in `price` tag in OpenAPI and gRPC communication to prevent price mis-precision
4. Only store hash encrypted password in DB instead of plain password
5. Prevent SQL injection in DB Service
6. Concurrency in DB Service using `Transaction` to prevent multi-thread conflict
7. Streaming gRPC for large log data across Logging Service and API Service.

We also do experiments on our API using `hey` to test the system QPS.

3. Setup

The setup is very easy since we use docker to compose everything.

To launch it, you can use the following command:

```
unzip 12211612_HaibinLai_A2.zip
cd codebase
```

```
# clean up and rebuild docker:
docker-compose down && docker-compose up --build -d

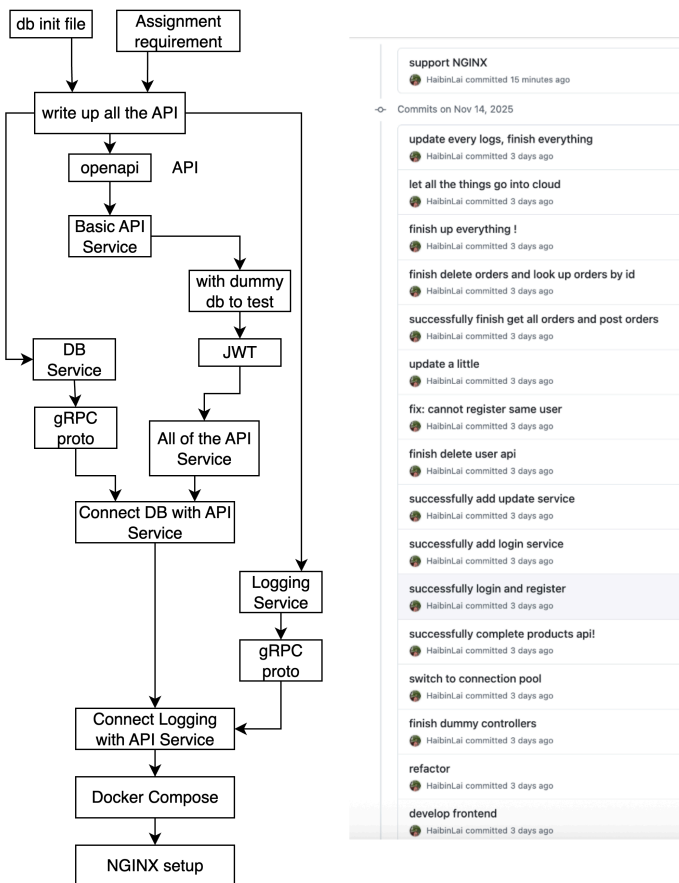
# to see logging service flow:
docker exec -it kafka kafka-console-consumer --bootstrap-server localhost:9092 --topic
api-logs --from-beginning
```

(If you encounter network problem, try to reconfig docker image source)

(Some python package may need to install)

How do I Implement the system?

In this section, I will explain how I set up the environment, generate code and implement the logic.



Environment setup

I prepared a working environment with:

- Python 3.10 with conda and pip management
- Flask / Connexion (OpenAPI-based Flask framework)
- Python gRPC tools (grpcio , grpcio-tools)
- PostgreSQL (via Docker)
- Kafka + Zookeeper (via Docker)
- config like DB password etc

```
sudo apt update
```

```
pip install confluent-kafka, psycpg2-binary, grpcio, grpcio-tools, protobuf,  
connexion, Flask, swagger-ui-bundle
```

```
# install PostgreSQL
```

```
sudo apt install postgresql postgresql-contrib
```

```
sudo systemctl status postgresql
```

```
sudo apt install docker-compose
```

```
# stop local pgsq
```

```
sudo systemctl stop postgresql
```

```
docker-compose down && docker-compose up --build -d
```

postgres docker compose:

Change your postgres database username, passwd and db config in docker `compose.yml` . So that you can build the images correctly.

```
services:
```

```
  # database for persistent data storage
```

```
  postgres:
```

```
    image: postgres:17
```

```
    container_name: postgres
```

```
    hostname: postgres
```

```
    environment:
```

```
      POSTGRES_USER: dncc # my username
```

```
      POSTGRES_PASSWORD: dncc # my passwd
```

```
      POSTGRES_DB: goodsstore
```

```
    ports:
```

```
      - "5432:5432" # Expose PostgreSQL to localhost ("local:container")
```

```
    volumes:
```

```
      - pg_data:/var/lib/postgresql/data
```

```
      - ./db-init:/docker-entrypoint-initdb.d
```

build up the docker image and run the instance:

```
docker-compose -f compose.yml down # clean the instance if possible
```

```
docker-compose -f compose.yml up --build -d # build up the images
```

API Decision

I break up the assignment requirement and generate a series of table to describe API just like:

RESTful API	Method	Description	Must input	Can be null	Return
/users/register	POST	Register	sid , password	username , email	SUCCESS/FAIL
/users/login	POST	Login	sid , password	-	SUCCESS/FAIL
/users/me	GET	Get my profile	JWT	-	Profile info
/users/me	PATCH	Update my profile	JWT, your update things	-	SUCCESS/FAIL
/users/me	DELETE	Delete an account	JWT	-	SUCCESS/FAIL

You can see the full of them in API Section.

OpenAPI+Flask

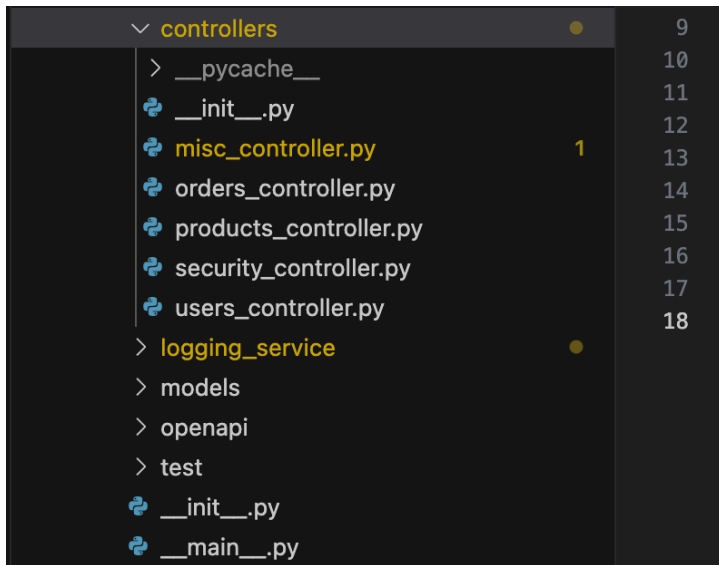
I write down the OpenAPI to finish the frontend API. And I use deepseek to detect if some of my code is wrong. Then I use generator to generate the code:

```
docker run --rm \
-v ${PWD}:/local \
openapitools/openapi-generator-cli:v7.8.0 generate \
-i /local/openapi.yaml \
-g python-flask \
-o /local/server/python-flask
```

```
(base) ccc@taskflow01:~/Distributed-and-Cloud-Computing/assignment2/codebase/src$ sudo docker run --rm -v "${PWD}:/local" openapitools/openapi-generator-cli:v7.8.0 generate -i /local/openapi.yaml -g python-flask -o /local/server/python-flask
stapi -o /local/api_server
[main] INFO o.o.c.l.AbstractPythonCodegen - Environment variable PYTHON_POST_PROCESS_FILE not defined so the Python code may not be properly formatted. To define it, try 'export PYTHON_POST_PROCESS_FILE="/usr/bin/python3 -m yaml" (Linux/Mac)
[main] INFO o.o.c.l.AbstractPythonCodegen - NOTE: To enable file post-processing, 'enablePostProcessFile' must be set to 'true' (--enable-post-process-file for CLI).
[main] WARN o.o.codegen.utils.ModelUtils - Failed to get the schema name: null
[main] INFO o.o.codegen.TemplateManager - writing file /local/api_server/src/openapi_server/models/greeting.py
[main] INFO o.o.codegen.TemplateManager - writing file /local/api_server/src/openapi_server/models/jwt_token.py
[main] INFO o.o.codegen.TemplateManager - writing file /local/api_server/src/openapi_server/models/login_req.py
[main] INFO o.o.codegen.TemplateManager - writing file /local/api_server/src/openapi_server/models/order.py
[main] INFO o.o.codegen.TemplateManager - writing file /local/api_server/src/openapi_server/models/place_order_req.py
[main] INFO o.o.codegen.TemplateManager - writing file /local/api_server/src/openapi_server/models/product.py
[main] INFO o.o.codegen.TemplateManager - writing file /local/api_server/src/openapi_server/models/register_user_req.py
[main] INFO o.o.codegen.TemplateManager - writing file /local/api_server/src/openapi_server/models/update_user_req.py
[main] INFO o.o.codegen.TemplateManager - writing file /local/api_server/src/openapi_server/models/user_public.py
[main] WARN o.o.codegen.DefaultCodegen - Empty operationId found for path: get /. Renamed to auto-generated operationId: rootGet
[main] WARN o.o.codegen.DefaultCodegen - Empty operationId found for path: get /products. Renamed to auto-generated operationId: productsGet
[main] WARN o.o.codegen.DefaultCodegen - Empty operationId found for path: get /products/{id}. Renamed to auto-generated operationId: productsIdGet
[main] WARN o.o.codegen.DefaultCodegen - Empty operationId found for path: post /users/register. Renamed to auto-generated operationId: usersRegisterPost
[main] WARN o.o.codegen.DefaultCodegen - Empty operationId found for path: post /users/login. Renamed to auto-generated operationId: usersLoginPost
[main] WARN o.o.codegen.DefaultCodegen - Empty operationId found for path: get /users/me. Renamed to auto-generated operationId: usersMeGet
[main] WARN o.o.codegen.DefaultCodegen - Empty operationId found for path: patch /users/me. Renamed to auto-generated operationId: usersMePatch
[main] WARN o.o.codegen.DefaultCodegen - Empty operationId found for path: post /users/deactivate. Renamed to auto-generated operationId: usersDeactivatePost
[main] WARN o.o.codegen.DefaultCodegen - Empty operationId found for path: get /orders/{id}. Renamed to auto-generated operationId: ordersIdGet
[main] WARN o.o.codegen.DefaultCodegen - Empty operationId found for path: delete /orders/{id}. Renamed to auto-generated operationId: ordersIdDelete
[main] INFO o.o.codegen.TemplateManager - writing file /local/api_server/src/openapi_server/apis/default_api.py
[main] INFO o.o.codegen.TemplateManager - writing file /local/api_server/src/openapi_server/apis/default_api_base.py
[main] INFO o.o.codegen.TemplateManager - writing file /local/api_server/tests/test_default_api.py
[main] INFO o.o.codegen.TemplateManager - writing file /local/api_server/README.md
[main] INFO o.o.codegen.TemplateManager - writing file /local/api_server/openapi.yaml
[main] INFO o.o.codegen.TemplateManager - writing file /local/api_server/src/openapi_server/main.py
[main] INFO o.o.codegen.TemplateManager - writing file /local/api_server/docker-compose.yaml
[main] INFO o.o.codegen.TemplateManager - writing file /local/api_server/Dockerfile
[main] INFO o.o.codegen.TemplateManager - writing file /local/api_server/requirements.txt
[main] INFO o.o.codegen.TemplateManager - writing file /local/api_server/src/openapi_server/security_api.py
[main] INFO o.o.codegen.TemplateManager - writing file /local/api_server/src/openapi_server/models/extra_models.py
[main] INFO o.o.codegen.TemplateManager - writing file /local/api_server/src/openapi_server/models/_init_.py
[main] INFO o.o.codegen.TemplateManager - writing file /local/api_server/src/openapi_server/apis/_init_.py
[main] INFO o.o.codegen.TemplateManager - writing file /local/api_server/src/openapi_server/impl/_init_.py
[main] INFO o.o.codegen.TemplateManager - writing file /local/api_server/tests/configtest.py
[main] INFO o.o.codegen.TemplateManager - writing file /local/api_server/.gitignore
[main] INFO o.o.codegen.TemplateManager - writing file /local/api_server/pyproject.toml
[main] INFO o.o.codegen.TemplateManager - writing file /local/api_server/setup.cfg
[main] INFO o.o.codegen.TemplateManager - writing file /local/api_server/.flake8
[main] INFO o.o.codegen.TemplateManager - writing file /local/api_server/.openapi-generator-ignore
[main] INFO o.o.codegen.TemplateManager - writing file /local/api_server/.openapi-generator/VERSION
[main] INFO o.o.codegen.TemplateManager - writing file /local/api_server/.openapi-generator/FILES
#####
# Thanks for using OpenAPI Generator.
# Please consider donation to help us maintain this project 🙏
# https://opencollective.com/openapi_generator/donate
#
# This generator's contributed by Nikita Vakula (https://github.com/kryakbrjak) #
# Please support his work directly via https://paypal.me/kryakbrjak 🙏
#####
(base) ccc@taskflow01:~/Distributed-and-Cloud-Computing/assignment2/codebase/src$ sudo docker run --rm -v "${PWD}:/local" openapitools/openapi-generator-cli:v7.8.0 generate -i /local/openapi.yaml -g python-flask -o /local/server/python-flask
stapi -o /local/api_server
C O A 11 W Q
```

Flask Controller+JWT

the FLASK has several controller for api service. Here I finish and test them. (If they need to communicate with db, just write dummy code first and impl them later)



For example, the order_get help users know their orders:

```
def orders_get(page: int = None, page_size: int = None):
    """
    GET /orders
    - Get user_id from JWT token
    - Take gRPC ListOrdersByUser
    """
    # Get info from JWT
    payload, err = _get_current_user_payload()
    if err is not None:
        return err

    user_id = payload.get("user_id")
    if user_id is None:
        # logging service, impl later
        logging_client.send_logs([
            {
                "service_name": "api-service/orders",
                "level": "ERROR",
                "path": connexion.request.path,
                "method": connexion.request.method,
                "user_sid": "",
                "message": "Invalid token: missing user_id"
            }
        ])
        return _error("Invalid token: missing user_id", 401)

    stub = _get_db_stub()

    req = db_pb2.ListOrdersByUserRequest(
        user_id=user_id,
        page=page or 1,
        page_size=page_size or 20,
```

```

)

try:
    resp = stub.ListOrdersByUser(req)
except grpc.RpcError as e:
    # logging service, impl later
    logging_client.send_logs([
        {
            "service_name": "api-service/orders",
            "level": "ERROR",
            "path": connexion.request.path,
            "method": connexion.request.method,
            "user_sid": "",
            "message": f"DB service error: {e.code().name} - {e.details()}"
        }
    ])
    return _error(f"DB service error: {e.code().name} - {e.details()}", 502)

orders = [_grpc_order_to_dict(o) for o in resp.orders]

# logging service, impl later
logging_client.send_logs([
    {
        "service_name": "api-service/orders",
        "level": "INFO",
        "path": connexion.request.path,
        "method": connexion.request.method,
        "user_sid": "",
        "message": f"Fetchd {len(orders)} orders for user {user_id}"
    }
])
return _success(
    "Orders fetched successfully",
    data={
        "items": orders,
        "page": resp.page,
        "page_size": resp.page_size,
        "total_count": resp.total_count,
    },
    code=200,
)

```

Here we use JWT to auth the userid.

JWT

The JWT will contain the `sid` and `user_id`

```

JWT_SECRET = os.environ.get("JWT_SECRET", "dev-secret")
JWT_ALGO = "HS256"
JWT_EXPIRE_HOURS = 12

def _get_current_user_payload():
    """
    Authorization: Bearer xxx -> JWT -> payload.

```

```

"""
auth_header = connexion.request.headers.get("Authorization", "")
if not auth_header.startswith("Bearer "):
    return None, _error("Missing or invalid Authorization header", 401)

token = auth_header.split(" ", 1)[1].strip()
try:
    payload = pyjwt.decode(token, JWT_SECRET, algorithms=[JWT_ALGO])
except pyjwt.ExpiredSignatureError:
    return None, _error("Token expired", 401)
except pyjwt.InvalidTokenError:
    return None, _error("Invalid token", 401)

return payload, None

```

DB Service

Serve using a connection pool:

```

def serve(port: int = 50051):
    logging.basicConfig(level=logging.INFO)

    init_pool()

    server = grpc.server(futures.ThreadPoolExecutor(max_workers=10))
    db_pb2_grpc.add_DbServiceServicer_to_server(DbService(), server)
    server.add_insecure_port(f"[::]:{port}")
    server.start()
    logging.info("DB gRPC server started on port %d", port)

    try:
        server.wait_for_termination()
    finally:
        global P00L
        if P00L is not None:
            P00L.closeall()
            P00L = None
            logging.info("DB connection pool closed.")

if __name__ == "__main__":
    serve()

```

We can have 10 connector at one time:

```

def init_pool():
    global P00L
    if P00L is not None:
        return
    dsn = os.environ.get("POSTGRES_DSN")

```

```

if not dsn:
    dsn = "dbname=goodsstore user=dncc password=dncc host=postgres port=5432"
logging.info(f"Initializing DB connection pool with DSN: {dsn}")
POOL = SimpleConnectionPool(minconn=1, maxconn=10, dsn=dsn)

def get_connection():
    if POOL is None:
        init_pool()
    return POOL.getconn()

def release_connection(conn):
    if POOL is not None:
        POOL.putconn(conn)
    else:
        conn.close()

```

Some of the CRUD example:

We use **Database Transaction** to make sure **concurrency**. with conn: wraps everything in one transaction. If any exception is raised inside, psycopg2 will automatically ROLLBACK when leaving the block. Any concurrent CreateOrder requests targeting the same product must wait until the current transaction finishes. Two customers cannot purchase the same remaining items at the same time, the stock value read from the database is always the latest and consistent, all subsequent updates to the product row happen safely. This is the standard technique for avoiding lost updates and overselling under PostgreSQL's default READ COMMITTED isolation level.

To prevent **SQL injection**, all database queries in the DB Service are executed using parameterized statements (e.g., WHERE sid = %s with (request.sid,) in psycopg2) instead of string concatenation. User inputs are always passed as bound parameters, so they are treated as data rather than executable SQL.

Use **Decimal** rather than float to compute the total price. And we also check the number and price that from DB.

```

# ---- Orders ----
def CreateOrder(self, request, context):
    """
    CreateOrder 逻辑说明 (在一个事务里完成):
    1. 使用 SELECT ... FOR UPDATE 锁定商品行, 读取 price 与 stock
    2. 校验 quantity 范围 (1~3) 以及库存是否足够
    3. 基于数据库中的 price 计算 total_price (使用 Decimal 处理金额)
    4. 向 orders 表插入新订单记录, 并返回生成的订单信息
    5. 扣减 products 表中的库存
    所有 SQL 操作都使用参数化查询 (%s + 参数元组), 防止 SQL 注入。
    价格只信任数据库中的 price, 不信任客户端传入的任何金额字段。
    """
    conn = get_connection()
    try:
        with conn: # commit / rollback

```

```

with conn.cursor() as cur:
    # 1. read stock & price
    cur.execute(
        """
        SELECT price, stock
        FROM products
        WHERE id = %s
        FOR UPDATE
        """,
        (request.product_id,),
    )
    prod_row = cur.fetchone()
    if prod_row is None:
        context.abort(grpc.StatusCode.NOT_FOUND, "product not found")

    price, stock = prod_row # price: NUMERIC -> Decimal, stock: int

    # 2. check quantity
    qty = int(request.quantity)
    if qty <= 0 or qty > 3:
        context.abort(
            grpc.StatusCode.INVALID_ARGUMENT,
            "quantity must be between 1 and 3",
        )

    if stock < qty:
        context.abort(
            grpc.StatusCode.FAILED_PRECONDITION,
            "not enough stock",
        )

    # 3. price, Decimal
    if not isinstance(price, Decimal):
        price = Decimal(str(price))
    total_price = price * Decimal(qty)

    # 4. insert
    cur.execute(
        """
        INSERT INTO orders (user_id, product_id, quantity, total_price)
        VALUES (%s, %s, %s, %s)
        RETURNING id, user_id, product_id, quantity, total_price,
        created_at
        """,
        (
            request.user_id,
            request.product_id,
            qty,
            total_price,
        ),
    )

```

```

        order_row = cur.fetchone()
        if order_row is None:
            context.abort(
                grpc.StatusCode.INTERNAL,
                "failed to create order",
            )

        # 5. minus stock
        cur.execute(
            """
            UPDATE products
            SET stock = stock - %s
            WHERE id = %s
            """,
            (qty, request.product_id),
        )
        # return
        return row_to_order(order_row)
    except Exception as e:
        conn.rollback()
    finally:
        release_connection(conn)

```

gRPC with API Service

gRPC provide all the CRUD service to Flask API Service:

```

service DbService {
    // Products
    rpc CreateProduct(CreateProductRequest) returns (Product);
    rpc GetProduct(GetProductRequest) returns (Product);
    rpc ListProducts(ListProductsRequest) returns (ListProductsResponse);
    rpc UpdateProduct(UpdateProductRequest) returns (Product);
    rpc DeleteProduct(DeleteProductRequest) returns (google.protobuf.Empty);

    // Users
    rpc CreateUser(CreateUserRequest) returns (User);
    rpc GetUserById(GetUserByIdRequest) returns (User);
    rpc GetUserBySid(GetUserBySidRequest) returns (User);
    rpc GetUserByUsername(GetUserByUsernameRequest) returns (User);
    rpc ListUsers(ListUsersRequest) returns (ListUsersResponse);
    rpc UpdateUser(UpdateUserRequest) returns (User);
    rpc DeleteUser(DeleteUserRequest) returns (google.protobuf.Empty);

    // Orders
    rpc CreateOrder(CreateOrderRequest) returns (Order);
    rpc GetOrder(GetOrderRequest) returns (Order);
    rpc ListOrders(ListOrdersRequest) returns (ListOrdersResponse);
    rpc ListOrdersByUser(ListOrdersByUserRequest) returns (ListOrdersResponse);
    rpc UpdateOrder(UpdateOrderRequest) returns (Order);
    rpc DeleteOrder(DeleteOrderRequest) returns (google.protobuf.Empty);
}

```

```
}
```

Take Orders as an example:

```
message CreateOrderRequest {
    int32 user_id = 1;
    int32 product_id = 2;
    int32 quantity = 3;
    // price is calculated by Database
}

message GetOrderRequest {
    int32 id = 1;
}

message ListOrdersRequest {
    int32 page = 1;
    int32 page_size = 2;
}

message ListOrdersByUserRequest {
    int32 user_id = 1;
    int32 page = 2;
    int32 page_size = 3;
}

message ListOrdersResponse {
    repeated Order orders = 1;
    int32 page = 2;
    int32 page_size = 3;
    int32 total_count = 4;
}

message UpdateOrderRequest {
    int32 id = 1;
    int32 quantity = 2;
}

message DeleteOrderRequest {
    int32 id = 1;
}
```

Generating the gRPC code with grpc tools:

```
python -m grpc_tools.protoc \
    -I. \
    --python_out=. \
```

```
--grpc_python_out=. \
db.proto logging.proto
```

The file will help communication. We use this code to test gRPC connection:

```
import grpc

import db_pb2 # two files generated by protoc
import db_pb2_grpc # two files generated by protoc

def main():
    channel = grpc.insecure_channel("localhost:50051")
    stub = db_pb2_grpc.DbServiceStub(channel)

    user = stub.CreateUser(
        db_pb2.CreateUserRequest(
            sid="12212345",
            username="haibin",
            email="laihb2022@mail.sustech.edu.cn",
            password_hash="dummy_hash", # create a user
        )
    )
    print("Created user:", user)
```

Then every code is used to connect gRPC like this template.

OpenAPI Server + gRPC with DB Server

Take Orders_id_get as an example:

```
def orders_id_get(id_, token_info=None):
    """Get a single order by id."""

    order_id = id_

    stub = _get_db_stub()
    try:
        req = db_pb2.GetOrderRequest(id=order_id)
        order_msg = stub.GetOrder(req)

        return _success("Order fetched", order_to_dict(order_msg), 200)

    except grpc.RpcError as e:
        if e.code() == grpc.StatusCode.NOT_FOUND:
            logging_client.send_logs([
                "service_name": "api-service/orders",
                "level": "WARNING",
                "path": connexion.request.path,
                "method": connexion.request.method,
                "user_sid": "",
```

```

        "message": "Order not found"
    })
    return _error("Order not found", 404)

logging_client.send_logs([
    {
        "service_name": "api-service/orders",
        "level": "ERROR",
        "path": connexion.request.path,
        "method": connexion.request.method,
        "user_sid": "",
        "message": f"DB service error: {e.code().name} - {e.details()}"
    }
])
return _error(
    f"DB service error: {e.code().name} - {e.details()}",
    502,
)

```

This handler is a good example of how my OpenAPI-based API Service integrates JWT auth, gRPC, and centralized logging in a layered way:

- **REST + OpenAPI layer**

`orders_id_get` is the controller for `GET /orders/{id}` generated from my OpenAPI spec. The `id_` parameter comes from the path, and `token_info` is injected by the **JWT bearerAuth** security handler. That means:

- The JWT is validated *before* this function runs.
- On success, decoded claims (e.g. `sid`, `user_id`) are available via `token_info`, even though this particular endpoint doesn't yet enforce per-user ownership.

- **gRPC DB Service layer**

Instead of talking to PostgreSQL directly, the API calls the **DB Service via gRPC**:

- `_get_db_stub()` returns a gRPC stub for the DB Service.
- The handler builds a strongly-typed `GetOrderRequest` (`db_pb2.GetOrderRequest(id=order_id)`) and calls `stub.GetOrder(req)`.
- On success, the protobuf `order_msg` is converted into a plain dict with `order_to_dict(order_msg)` and wrapped by `_success("Order fetched", ..., 200)` as a JSON HTTP 200 response.
- This cleanly separates HTTP/JSON concerns (API Service) from SQL/connection-pooling concerns (DB Service).

- **Error mapping and service “tiering”**

gRPC status codes from the DB Service are mapped to appropriate HTTP responses at the API tier:

- If `e.code() == grpc.StatusCode.NOT_FOUND`, the API returns an HTTP **404** "Order not found".
- Any other gRPC error is treated as an internal backend failure and returned as HTTP **502** "DB service error: ...".
- This keeps the REST interface stable while the DB tier is an internal implementation detail.

- **Centralized structured logging via Logging Service + Kafka**

On every failure, the handler sends a **structured log** to the Logging Service client:

- Fields include `service_name` ("api-service/orders"), `log_level` ("WARNING" or "ERROR"), HTTP path , method , and a human-readable message .
- These logs are streamed via gRPC to the Logging Service, which then forwards them into Kafka, so an administrator can monitor failed `get-order` calls centrally.
- The `user_sid` field is already reserved; once I plug in `token_info` , I can attach the authenticated user's SID to each log entry.

Overall, this endpoint shows my backend design goals:

- **JWT-protected OpenAPI REST layer** (with `token_info` plumbed in),
- **A middle gRPC tier (DB Service)** that encapsulates all database logic and error codes, outputting status code and detail message ,
- **A separate Logging Service** that receives structured logs from the API Service and forwards them to Kafka for monitoring.

Logging Service + gRPC

1. write the code for logs

```

44
45 class LoggingServiceServicer(object):
46     """注意: 这是 **client-side streaming**
47     """
48
49     def StreamLogs(self, request_iterator, context):
50         """Missing associated documentation comment in .proto file."""
51         context.set_code(grpc.StatusCode.UNIMPLEMENTED)
52         context.set_details('Method not implemented!')
53         raise NotImplementedError('Method not implemented!')
54
55
56 def add_LoggingServiceServicer_to_server(servicer, server):
57     rpc_method_handlers = {
58         'StreamLogs': grpc.stream_unary_rpc_method_handler(
59             servicer.StreamLogs,
60             request_deserializer=logging__pb2.LogMessage.FromString,
61             response_serializer=logging__pb2.LogSummary.SerializeToString,
62         ),
63     }
64     generic_handler = grpc.method_handlers_generic_handler(
65         'logging.LoggingService', rpc_method_handlers)
66     server.add_generic_rpc_handlers((generic_handler,))
67     server.add_registered_method_handlers('logging.LoggingService', rpc_method_handlers)
68

```

Then use gRPC to communicate.

```

syntax = "proto3";

package logging;

option go_package = "loggingpb";

message LogMessage {
    string service_name = 1;    // "api-service" / "db-service" 等
    string level         = 2;    // "INFO" / "WARN" / "ERROR"
    string path          = 3;    // 请求路径, 比如 "/users/login"
    string method        = 4;    // "GET" / "POST"
    string user_sid      = 5;    // 可选: 当前用户 SID
    string message       = 6;    // 文本日志, 比如 "User login success"
}

```

```

int64 timestamp_ms = 7;    // 客户端填: 毫秒级 Unix 时间戳
}

message LogSummary {
  int64 count = 1; // num of logs
}

// **client-side streaming**
service LoggingService {
  rpc StreamLogs (stream LogMessage) returns (LogSummary);
}

```

`send_logs` is the function we use in all the API Service.

```

def send_logs(logs):
    """
    logs: List[dict], example:
        {
            "service_name": "api-service",
            "level": "INFO",
            "path": "/users/login",
            "method": "POST",
            "user_sid": "123",
            "message": "login ok"
        }
    """
    stub = get_logging_stub()

    INSTANCE_NAME = os.getenv("INSTANCE_NAME", "unknown")

    def gen():
        for log in logs:
            yield logging_pb2.LogMessage(
                service_name=INSTANCE_NAME,
                level=log.get("level", "INFO"),
                path=log.get("path", ""),
                method=log.get("method", ""),
                user_sid=log.get("user_sid", ""),
                message=log.get("message", ""),
                timestamp_ms=int(time.time() * 1000)
            )

    return stub.StreamLogs(gen())

```

Then the API Service can use:

```

def root_get(): # noqa: E501
    """Greeting
    Return a welcome message. # noqa: E501
    :rtype: Union[str, Tuple[str, int], Tuple[str, int, Dict[str, str]]

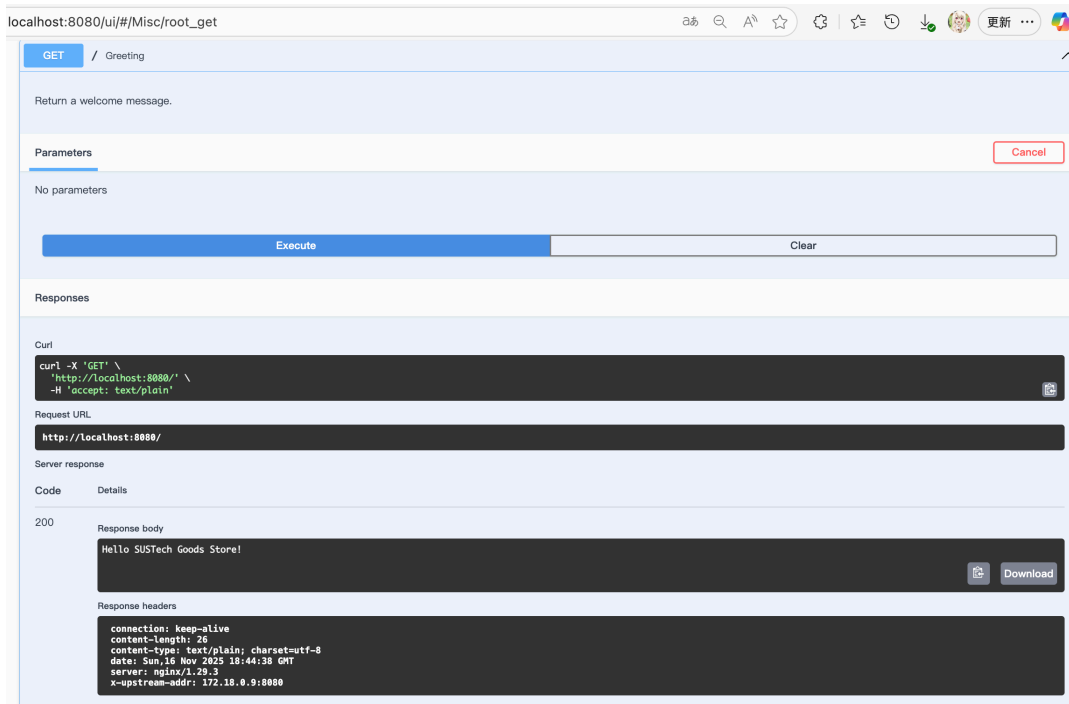
```

```

logging_client.send_logs([
    "service_name": "api-service/misc",
    "level": "INFO",
    "path": "/",
    "method": "GET",
    "user_sid": "",
    "message": "Root endpoint accessed"
])

return 'Hello SUSTech Goods Store!'

```



```

may not be available. (org.apache.kafka.clients.NetworkClient)
(base) cgetaskflow@it-01distributed-and-cloud-computing/assignment2/codebase$ docker exec -it kafka kafka-console-consumer --bootstrap-server localhost:9092 --topic api-logs
[2025-11-16 17:04:42.389] WARN [Consumer clientId=console-consumer, groupId=console-consumer-28677] Error while fetching metadata with correlation id 2 : {api-logs=LEADER_NOT_AVAILABLE} (org.apache.kafka.clients.NetworkClient)
{"service_name": "api1", "level": "INFO", "path": "/products", "method": "GET", "user_sid": "", "message": "Products fetched: 3", "timestamp_ms": 1763312687848}
{"service_name": "api2", "level": "INFO", "path": "/products/1", "method": "GET", "user_sid": "", "message": "Product 1 fetched", "timestamp_ms": 1763312695605}
{"service_name": "api1", "level": "INFO", "path": "/products/1", "method": "GET", "user_sid": "", "message": "Product 1 fetched", "timestamp_ms": 1763312696835}
{"service_name": "api2", "level": "INFO", "path": "/products/1", "method": "GET", "user_sid": "", "message": "Product 1 fetched", "timestamp_ms": 1763312697637}

```

you can also observe the load imbalance here (The api1 and api2 are two API Server in docker network)

Docker Compose Network

Connect everything in the docker

```
##### YOUR SERVICES GO BELOW THIS LINE #####
```

```

db_service:
  build: ./src/db_service
  container_name: db_service
  depends_on:
    - postgres
  environment:

```

```
POSTGRES_DSN: "dbname=goodsstore user=dncc password=dncc host=postgres
port=5432"
```

```
# ----- API 副本 -----
api1:
  build:
    context: ./src/api_service/server/python-flask/
  container_name: api1
  env_file:
    - .env
  environment:
    - LOGGING_ADDR=logging_service:50052
    - INSTANCE_NAME=api1
    - DB_ADDR=db_service:50051
  ports: [] # 不再直接映射到宿主机端口, 由 nginx 代理
  depends_on:
    - db_service
    - logging_service

api2:
  build:
    context: ./src/api_service/server/python-flask/
  container_name: api2
  env_file:
    - .env
  environment:
    - INSTANCE_NAME=api2
    - LOGGING_ADDR=logging_service:50052
    - DB_ADDR=db_service:50051
  ports: []
  depends_on:
    - db_service
    - logging_service

# ----- NGINX -----
nginx:
  image: nginx:alpine
  container_name: nginx_lb
  depends_on:
    - api1
    - api2
  ports:
    - "8080:80" # http://localhost:8080 -> NGINX
  volumes:
    - ./nginx.conf:/etc/nginx/nginx.conf:ro
  networks:
    - default

logging_service:
  build:
    context: ./src/logging_service
```

```
    dockerfile: Dockerfile
    container_name: logging_service
    hostname: logging_service
    depends_on:
      - kafka
    environment:
      - KAFKA_BOOTSTRAP_SERVERS=kafka:9092
      - KAFKA_TOPIC=api-logs
    # gRPC internel
    ports:
      - "50052:50052"
    networks:
      - default

volumes:
  pg_data: # A placeholder volume without any configuration
```

4. API Design

In this section, I will show the API Design and authentication logic.

We have 3 kinds of api in the service:

1. Greeting and See products API: the API to check the website.
2. User accounts API: the API for user accounts like login, delete and update profile.
3. Orders API: the API to check and order users' orders.

All of the api are shown here:

Misc					
GET	/	Greeting			
Orders					
GET	/orders	List all orders of current user			
POST	/orders	Place an order			
DELETE	/orders/{id}	Cancel an order			
GET	/orders/{id}	Get an order by id (current user's order)			
Products					
GET	/products	List products			
GET	/products/{id}	Get product by id			
Users					
POST	/users/login	Login and get JWT			
DELETE	/users/me	Deactivate current user's account			
GET	/users/me	Get current user's profile			
PATCH	/users/me	Update current user's profile			
POST	/users/register	Register a new user			

At the base URL, user can find a greeting API that returns a welcome message. And they can check the products without any password or JWT.

RESTful API	Method	Description	Need Passwd?	Need JWT?	Return
/	GET	Greeting	✗	✗	"Hello!"
/products	GET	List products	✗	✗	All Products
/products/{id}	GET	Get product	✗	✗	Product

All of them don't need authentication, since they should be the goods and page that show to everyone that visit the online shop.

A product information contains:

Feature	Data type
id	key int
Name	string
Description	string

Feature	Data type
Category	string
price	string
slogan	string
stock	int
created_at	time

Note: here in design, we should use string or decimal to design `price` tag. The OpenAPI only have string, so we design it into string. And in database, we use decimal to calculate the total price:

```
from decimal import Decimal

price = Decimal(row.price)          # DB → Decimal
qty = Decimal(request.quantity)
total = price * qty
```

To place orders, the client have to register an account in the system. The following API can help them deal with their account information with `register`, `login`, `getting profile`, `update profile` and `delete account` :

RESTful API	Method	Description	Need Passwd?	Need JWT?
/users/register	POST	Register	✓	✗
/users/login	POST	Login	✓	✗ (login will generate JWT)
/users/me	GET	Get my profile	✗	✓
/users/me	PATCH	Update my profile	✗	✓
/users/me	DELETE	Delete an account	✗	✓

The getting profile, updating profile and deleting account needs JWT to authentication. And Login & Register need password.

Every user can see their `Profile info` :

User	Data type	Can be seen?	Changeable?
id	int	✓	✗
username	string	✓	✓
email	string	✓	✓
sid	string	✓	✗
created_at	time	✓	✗
password	string	✗	✓ (Change hash in db)

The password will be encrypted with hash when sending to database:

```
password_hash = hashlib.sha256(password.encode("utf-8")).hexdigest()
try:
    req = db_pb2.CreateUserRequest(
        sid=sid,
        username=username,
        email=email,
        password_hash=password_hash,
    )
    user = stub.CreateUser(req)
```

So that the db won't leak user's password.

And this is the input and return for the user api:

RESTful API	Method	Description	Must input	Can be null	Return
/users/register	POST	Register	sid , password	username , email	SUCCESS/FAIL
/users/login	POST	Login	sid , password	-	SUCCESS/FAIL
/users/me	GET	Get my profile	JWT	-	Profile info
/users/me	PATCH	Update my profile	JWT, your update things	-	SUCCESS/FAIL
/users/me	DELETE	Delete an account	JWT	-	SUCCESS/FAIL

After having a complete account, they can place orders, check their own orders and delete one of their orders by using the following API:

RESTful API	Method	Description	Need Passwd?	Need JWT?
/orders	POST	Place an order	✗	✓
/orders/{id}	GET	Check your own order with id	✗	✓
/orders	GET	Check all your own orders	✗	✓
/orders/{id}	DELETE	Cancel an order	✗	✓

Product	Data type	Can be seen?
id	int	✓
userid	int	✓
product_id	int	✓
quantity	int	✓
created_at	time	✓

Product	Data type	Can be seen?
total_price	string	✓

RESTful API	Method	Description	Must input	Return
/orders	POST	Place an order	product_id , quantity	total_price , created_at
/orders/{id}	GET	Check order with id	id , JWT	Your Order info
/orders/	GET	Check all your order	JWT	Your Orders info
/orders/{id}	DELETE	Cancel an order	JWT	SUCCESS/FAIL

Generate:

```
docker run --rm \
  -v ${PWD}:/local \
  openapitools/openapi-generator-cli:v7.8.0 generate \
  -i /local/openapi.yaml \
  -g python-flask \
  -o /local/server/python-flask
```

In my API Service, **all “user-specific” operations require authentication**, while read-only “public” operations don’t.

4.1 Which APIs require authentication?

No authentication required (public):

- GET / – Greeting API
- GET /products – List all products
- GET /products/{id} – Get product details
- POST /users/register – Create a new user account
- POST /users/login – User login, returns a JWT

Authentication required (JWT Bearer):

- **User APIs**
 - GET /users/me – Get the current user’s profile
 - GET /users/{id} – Get a specific user (only if it’s the same as the logged-in user)
 - PUT /users/{id} – Update user info
 - DELETE /users/{id} – Deactivate user
- **Order APIs**

- `POST /orders` – Place a new order (place-order)
- `GET /orders/{id}` – Get order details (only owner can see it)
- `DELETE /orders/{id}` – Cancel an order

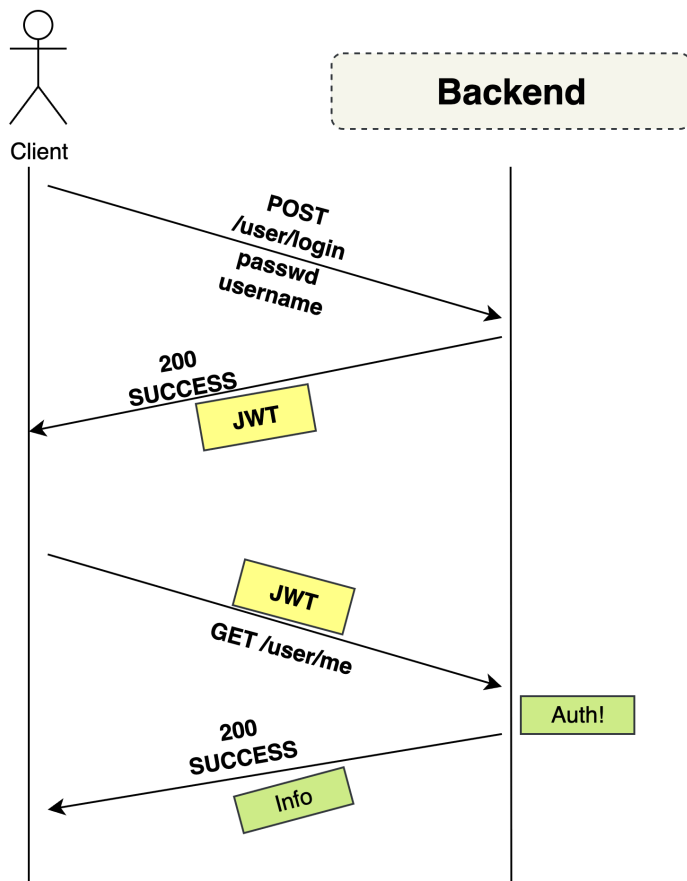
In the OpenAPI YAML, these operations are marked with a `security` section using a `bearerAuth` security scheme, so **Swagger UI, connexion, etc. know they require a token.**

4.2 How I implement the authentication logic

To create a JWT, clients first call the `POST /users/login` API with a username and password. The API Service uses gRPC to query the DB Service for the user record and verifies the password against a stored hash. On success, the API Service issues a JWT signed with a secret key loaded from the environment. The JWT payload contains the user's `sid`, `user_id`, and an expiration timestamp.

In our API service, user-specific operations such as fetching the current user profile (`GET /users/me`), updating/deactivating the current user, and creating / listing / cancelling orders are protected by JWT-based authentication.

For subsequent requests, the client includes this token in the `Authorization: Bearer <token>` HTTP header. The API Service validates the token on each protected endpoint by verifying its signature and expiration. It then extracts the `sid` from the payload and calls the DB Service (e.g., `GetUserBySid`) to retrieve the authoritative user record and `user_id`. All downstream gRPC calls use this `user_id` instead of trusting any client-supplied identifier, so users can only access their own orders or profile. Additionally, every login and order-related request is logged via a client-side streaming gRPC connection to the Logging Service, which forwards all log messages to a Kafka topic for centralized monitoring.



4.2.1. Security scheme in OpenAPI

In the OpenAPI spec I define a JWT bearer scheme:

```
components:
  securitySchemes:
    bearerAuth:
      type: http
      scheme: bearer
      bearerFormat: JWT
```

Then for each protected operation:

```
paths:
  /orders:
    post:
      security:
        - bearerAuth: []
      ...
  /users/me:
    get:
      security:
        - bearerAuth: []
      ...
```

This tells connexion to call my `info_from_bearerAuth` function for these APIs.

4.2.2. Issuing the JWT on login

When the client calls `POST /users/login` with email/password:

1. API Service calls the **DB Service** via gRPC to fetch the user by id
2. It verifies the password against the stored hash.
3. If valid, it creates a JWT, for example:

```
import jwt
from datetime import datetime, timedelta

JWT_SECRET = os.getenv("JWT_SECRET", "dev_secret")
JWT_ALG = "HS256"
JWT_EXPIRE_HOURS = 12

def _create_jwt_for_user(u: db_pb2.User) -> str:
    now = datetime.now(timezone.utc)
    payload = {
        "sid": u.sid,
        "user_id": u.id,
        "iat": int(now.timestamp()),
        "exp": int((now + timedelta(hours=JWT_EXPIRE_HOURS)).timestamp()),
    }
    token = pyjwt.encode(payload, JWT_SECRET, algorithm=JWT_ALGO)
    # PyJWT>=2 return str
    return token
```

4. The login API returns:

localhost:8080/ui/#/Users/users_login_post

Request body *required*

application/json

```
{
  "password": "123",
  "sid": "123"
}
```

Execute Clear

Responses

Curl

```
curl -X 'POST' \
  'http://localhost:8080/users/login' \
  -H 'accept: application/json' \
  -H 'Content-Type: application/json' \
  -d '{
    "password": "123",
    "sid": "123"
  }'
```

Request URL

http://localhost:8080/users/login

Server response

Code Details

200

Response body

```
{
  "code": 200,
  "data": {
    "token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJzdWIiOiIxMjM1IiwiaWF0IjE2ODV9.8akvMEJvNjYzRR924eboTc3pFZkx5SM998JszRgUis",
    "user": {
      "created_at": "2025-11-16T18:57:48.408615+00:00",
      "email": "123@2",
      "is_active": true,
      "sid": "123",
      "username": "123"
    }
  },
  "message": "Login successful"
}
```

Download

The client then sends this token in the Authorization HTTP header:

Authorization: Bearer <JWT>.

And they can get the account info:

localhost:8080/ui/#/Users/users_me_get

POST /users/Login Login and get JWT

DELETE /users/me Deactivate current user's account

GET /users/me Get current user's profile

Parameters

No parameters

Execute Clear

Responses

Curl

```
curl -X 'GET' \
  'http://localhost:8080/users/me' \
  -H 'accept: application/json' \
  -H 'Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJzdWIiOiIxMjM1IiwiaWF0IjE2ODV9.8akvMEJvNjYzRR924eboTc3pFZkx5SM998JszRgUis'
```

Request URL

http://localhost:8080/users/me

Server response

Code Details

200

Response body

```
{
  "code": 200,
  "data": {
    "created_at": "2025-11-16T18:57:48.408615+00:00",
    "email": "123@2",
    "is_active": true,
    "sid": "123",
    "username": "123"
  },
  "message": "User profile fetched"
}
```

Response headers

```
connection: keep-alive
content-length: 211
content-type: application/json
date: Sun, 16 Nov 2025 18:58:45 GMT
server: nginx/1.29.3
x-upstream-addr: 172.18.0.8:8080
```

localhost:8080/ui/#/orders/orders_post

Execute

Clear

Responses

Curl

```
curl -X 'POST' \
  http://localhost:8080/orders' \
  -H 'accept: application/json' \
  -H 'Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJzdWIjOiIxMjMjc1NjZyXzkiLCJpcxoxdWludF07IjozeHMzMDQ1LjE3NCJmZWNoIjE3NDY5LjA4eVMEJmV9ZRR9Z4boTc3pPF2x_5SM998Js2RgUIs' \
  -H 'Content-Type: application/json' \
  -d '{
    "product_id": 1,
    "quantity": 2
  }'
```

Request URL

```
http://localhost:8080/orders
```

Server response

Code	Details
201	Response body<pre>{ "code": 201, "data": { "created_at": "2025-11-16T19:01:50.000160", "id": 1, "product_id": 1, "quantity": 2, "total_price": 99.98, "user_id": 10 }, "message": "Order created successfully" }</pre> Response headers<pre>connection: keep-alive content-length: 224 content-type: application/json date: Sun, 16 Nov 2025 19:01:50 GMT server: nginx/1.29.3 x-upstream-addr: 172.18.0.0:8080</pre>

Responses

Connexion calls `info_from_bearerAuth(token)` for any operation that requires `bearerAuth`. My implementation roughly looks like:

```
import jwt
from jwt import ExpiredSignatureError, InvalidTokenError
from openapi_server.models.error_response import ErrorResponse

JWT_SECRET = os.getenv("JWT_SECRET", "dev_secret")
JWT_ALG = "HS256"

def info_from_bearerAuth(token: str):
    """
    Check and retrieve authentication information from custom bearer token.
    Returned value will be passed in 'token_info' parameter of your operation
    function, if there is one.
    'sub' or 'uid' will be set in 'user' parameter of your operation function, if
    there is one.

    :param token: Token provided by Authorization header (without 'Bearer ')
    :type token: str
    :return: Decoded token information or None if token is invalid
    :rtype: dict | None
    """
```

```

"""
try:
    # 解 JWT, 验证签名和 exp
    payload = jwt.decode(token, JWT_SECRET, algorithms=[JWT_ALGORITHM])
    # 你登录生成里放了 sid 和 user_id, 这里直接返回整个 payload
    return payload
except ExpiredSignatureError:
    # 过期
    print("[JWT] Token expired")
    return None
except InvalidTokenError as e:
    # 签名错误 / 格式错误 / 算法不对等都进这里
    print(f"[JWT] Invalid token: {e}")
    return None

```

Now, the controller functions can accept `token_info` :

```

def users_me_get() -> Tuple[Dict, int]:
    """
    获取当前登录用户信息:
    - 从 JWT 拿 user_id 或 sid
    - 再调 gRPC GetUserBySid / GetUserById
    """
    payload, err = _get_current_user_payload()
    if err is not None:
        return err

    stub = _get_db_stub()
    sid = payload.get("sid")

    try:
        req = db_pb2.GetUserBySidRequest(sid=sid)
        user = stub.GetUserBySid(req)
    except grpc.RpcError as e:
        if e.code() == grpc.StatusCode.NOT_FOUND:
            logging_client.send_logs([
                {
                    "service_name": "api-service/user",
                    "level": "WARNING",
                    "path": connexion.request.path,
                    "method": connexion.request.method,
                    "user_sid": sid or "",
                    "message": "User not found"
                }
            ])
            return _error("User not found", 404)
    logging_client.send_logs([
        {
            "service_name": "api-service/user",
            "level": "ERROR",
            "path": connexion.request.path,
            "method": connexion.request.method,
            "user_sid": sid or "",
            "message": f"DB service error: {e.code().name} - {e.details()}"
        }
    ])

```

```

    })
    return _error(f"DB service error: {e.code().name} - {e.details()}", 502)

logging_client.send_logs([
    "service_name": "api-service/user",
    "level": "INFO",
    "path": connexion.request.path,
    "method": connexion.request.method,
    "user_sid": sid,
    "message": "User profile fetched successfully"
])
return _success("User profile fetched", _grpc_user_to_profile(user), 200)

```

So **authentication** is: “verify JWT, extract user info”.

And I also add a simple **authorization** check: the `user_id` in the token must match the resource’s owner (`order.user_id`, `user.id`, etc.).

4.2.4. Error behavior

- If the Authorization header is missing, malformed, or the token is invalid/expired, the API returns **401 Unauthorized** with a JSON error body.

```

$ curl -X 'POST' \
  'http://localhost:8080/orders' \
  -H 'accept: application/json' \
  -H 'Content-Type: application/json' \
  -d '{
    "product_id": 1,
    "quantity": 2
  }'
{
  "detail": "No authorization token provided",
  "status": 401,
  "title": "Unauthorized",
  "type": "about:blank"
}
curl: (3) URL rejected: Malformed input to a URL function
(base) haibin@haibindeMacBook-Pro ~ %

```

- If the token is valid, but the user tries to access another user’s data / order, the API returns **403 Forbidden**.
 - Other code for different reasons as shown above (404, 502 etc).
-

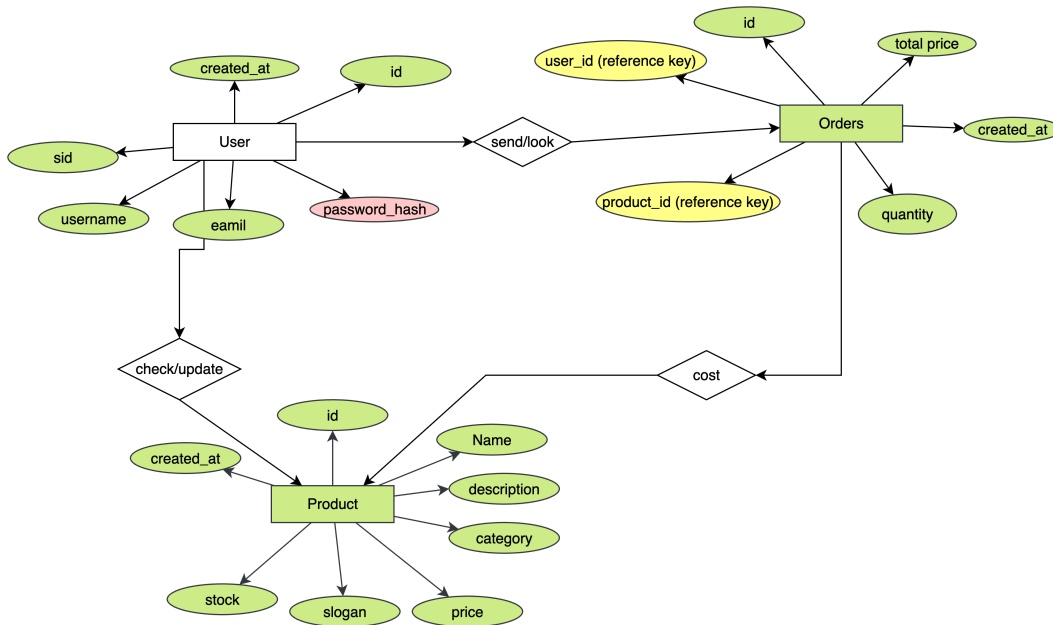
So in summary:

- I expose a **public subset** of APIs (greeting, product browsing, register, login).
- All **user-specific operations and order operations** are protected with JWT bearer authentication (`bearerAuth` in OpenAPI).
- The auth logic is implemented via **PyJWT** + an `info_from_bearerAuth` hook in connexion: login issues a signed, expiring JWT; each protected call decodes the token, checks expiry, and then uses the embedded `user_id` / `sid` to fetch data from the DB Service and enforce ownership.

5. Field Data types

This section shows how I select data types. Mostly I decide them base on database data type. And we show our principle and an example of product .

First, I drew an E-R graph for the DB:



This is the most important part that I get understand the whole structure. By understanding the relationship, we can choose and select the data type with full consideration.

📌 General Selection Principles

1. Use database-native types first

PostgreSQL has rich numeric and timestamp types, so DB types are chosen based on data characteristics.

2. Choose Protobuf types that safely map to DB types

Protobuf has fewer scalar types; for example:

- No decimal → use `string` or `int64` scaled representation
- No datetime → use `google.protobuf.Timestamp`

3. OpenAPI must match the Proto type semantics

While JSON has limited primitives, it must align with Protobuf encoding.

Example: Products

Feature	Data type
id	key int
Name	string
Description	string
Category	string

Feature	Data type
price	string/Decimal
slogan	string
stock	int
created_at	time

We store product in DB as a table:

```
CREATE TABLE products (
  id SERIAL PRIMARY KEY,
  name VARCHAR(100) NOT NULL,
  description TEXT,
  category VARCHAR(50),
  price DECIMAL(10, 2) NOT NULL,
  slogan VARCHAR(255),
  stock INT NOT NULL DEFAULT 500,
  created_at TIMESTAMP DEFAULT NOW()
);
```

Then for gRPC proto:

```
message Product {
  int32 id = 1;
  string name = 2;
  string description = 3;
  string category = 4;
  string price = 5;
  string slogan = 6;
  int32 stock = 7;
  google.protobuf.Timestamp created_at = 8;
}
```

- int32 for id and stock → safe match with PostgreSQL INTEGER
- string price → because protobuf has **no decimal** type
 - Using string prevents rounding errors
 - Matches what psycopg2 returns for NUMERIC

As for OpenAPI, we also use similar data type, so that OpenAPI can collaborate with gRPC very well.

```
Product:
  type: object
  properties:
    id:
      type: integer
      example: 1
    name:
```

```
    type: string
    example: "Laptop"
description:
  type: string
  example: "A powerful laptop"
category:
  type: string
  example: "Electronics"
price:
  type: string
  example: "999.99"
slogan:
  type: string
  example: "Fast and Light"
stock:
  type: integer
  example: 10
created_at:
  type: string
  format: date-time
  example: "2025-11-13T12:00:00Z"
required:
  - id
  - name
  - price
  - stock
  - created_at
```

This is a simple summary of our example.

Field	PostgreSQL Type	Proto Type	OpenAPI Type	Reason
id	SERIAL / INTEGER	int32	integer(int32)	Simple numeric key
name	TEXT	string	string	Textual data
category	TEXT	string	string	Textual data
price	NUMERIC(10,2)	string	string	Avoids float precision loss
stock	INTEGER	int32	integer(int32)	Integer count

6. gRPC verification

In this section we try to do gRPC verification just like things in our lab.

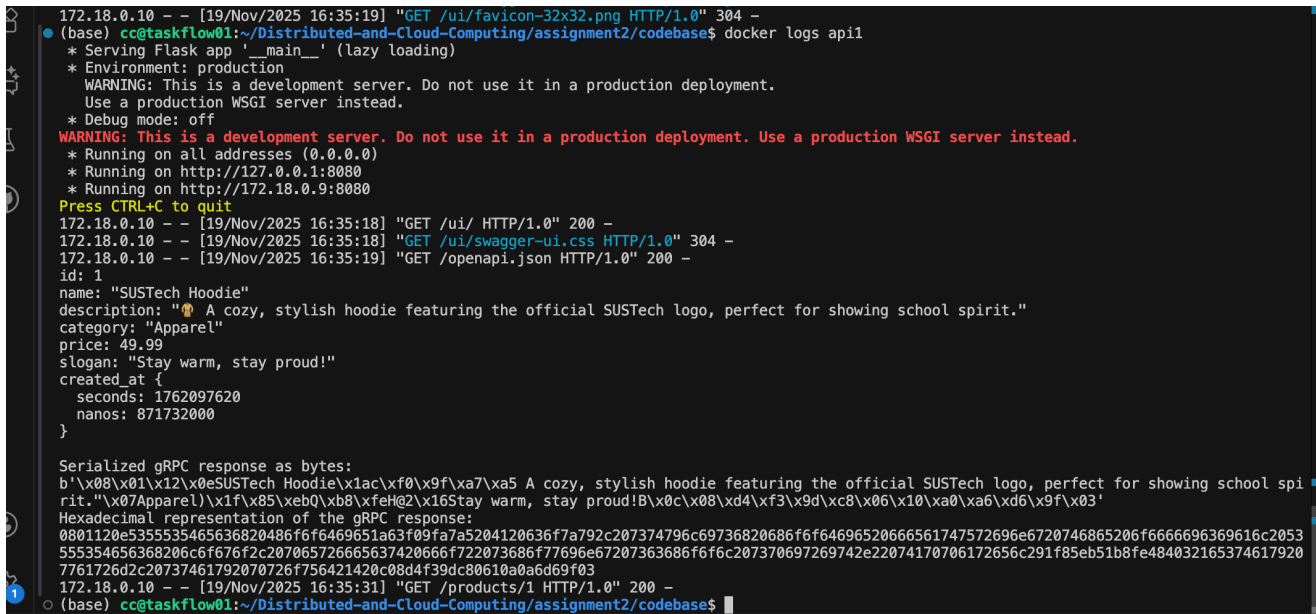
We try to print out the Hex strings:

```
try:
    resp = stub.ListProducts(req)
except:
    pass
```

```
# print out resp as gRPC message for debugging
res2 = resp.SerializeToString()
print(resp, flush=True)
print(res2, flush=True)

import binascii
hex_str = binascii.hexlify(res2).decode("ascii")
print("Hexadecimal representation of the gRPC response:", flush=True)
print(hex_str, flush=True)
```

Here is the code:



```
172.18.0.10 - - [19/Nov/2025 16:35:19] "GET /ui/favicon-32x32.png HTTP/1.0" 304 -
(base) cc@taskflow01:~/Distributed-and-Cloud-Computing/assignment2/codebase$ docker logs api1
* Serving Flask app '__main__' (lazy loading)
* Environment: production
  WARNING: This is a development server. Do not use it in a production deployment.
  Use a production WSGI server instead.
* Debug mode: off
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
* Running on all addresses (0.0.0.0)
* Running on http://127.0.0.1:8080
* Running on http://172.18.0.9:8080
Press CTRL+C to quit
172.18.0.10 - - [19/Nov/2025 16:35:18] "GET /ui/ HTTP/1.0" 200 -
172.18.0.10 - - [19/Nov/2025 16:35:18] "GET /ui/swagger-ui.css HTTP/1.0" 304 -
172.18.0.10 - - [19/Nov/2025 16:35:19] "GET /openapi.json HTTP/1.0" 200 -
id: 1
name: "SUSTech Hoodie"
description: "🧥 A cozy, stylish hoodie featuring the official SUSTech logo, perfect for showing school spirit."
category: "Apparel"
price: 49.99
slogan: "Stay warm, stay proud!"
created_at {
  seconds: 1762097620
  nanos: 871732000
}

Serialized gRPC response as bytes:
b'\x08\x01\x12\x0eSUSTech Hoodie\x1a\x0f\x0f\xa7\xa5 A cozy, stylish hoodie featuring the official SUSTech logo, perfect for showing school spi
rit.\x07Apparel)\x1f\x85\xebQ\xb8\xfeH\x16Stay warm, stay proud!B\x0c\x08\xd4\xf3\x9d\xc8\x06\x10\xa0\xa6\xd6\x9f\x03'
Hexadecimal representation of the gRPC response:
0801120e5355535465636820486f6f6469651a63f09fa7a5204120636f7a792c207374796c69736820686f6f64696520666561747572696e6720746865206f666666696369616c2053
555354656368206c6f6762c2070657266656374206666f722073686f77696e67207363686f66c207370697269742e22074170706172656c291f85eb51b8fe484032165374617920
7761726d2c20737461792070726f756421420c08d4f39dc80610a0a6d69f03
172.18.0.10 - - [19/Nov/2025 16:35:31] "GET /products/1 HTTP/1.0" 200 -
(base) cc@taskflow01:~/Distributed-and-Cloud-Computing/assignment2/codebase$
```

So this is our message, and our encrypted code:

```
id: 1
name: "SUSTech Hoodie"
description: "🧥 A cozy, stylish hoodie featuring the official SUSTech logo, perfect
for showing school spirit."
category: "Apparel"
price: 49.99
slogan: "Stay warm, stay proud!"
created_at {
  seconds: 1762097620
  nanos: 871732000
}
```

JSON length: 289

Serialized gRPC response as bytes:

```
b'\x08\x01\x12\x0eSUSTech Hoodie\x1a\x0f\x0f\xa7\xa5 A cozy, stylish hoodie featuring
the official SUSTech logo, perfect for showing school
```

```
spirit."\x07Apparel)\x1f\x85\xebQ\xb8\xfeH@2\x16Stay warm, stay  
proud!B\x0c\x08\xd4\xf3\x9d\xc8\x06\x10\xa0\xa6\xd6\x9f\x03'
```

Hexadecimal representation of the gRPC response:

```
0801120e5355535465636820486f6f6469651a63f09fa7a5204120636f7a792c207374796c69736820686f  
6f64696520666561747572696e6720746865206f6666696369616c2053555354656368206c6f676f2c2070  
65726665637420666f722073686f77696e67207363686f6f6c207370697269742e22074170706172656c29  
1f85eb51b8fe4840321653746179207761726d2c20737461792070726f756421420c08d4f39dc80610a0a6  
d69f03
```

That's 175 byte

--- Size Comparison ---

Protobuf : 175 bytes

JSON : 289 bytes

Ratio : 0.606

We can use following code to generate decode result:

```
import binascii

# -----
# Wire type constants
# -----
WIRE_VARINT = 0
WIRE_64BIT = 1
WIRE_LENGTH_DELIMITED = 2

# -----
# Helper: decode a varint
# -----
def read_varint(buffer, index):
    shift = 0
    result = 0
    while True:
        b = buffer[index]
        index += 1
        result |= (b & 0x7F) << shift
        if not (b & 0x80):
            break
        shift += 7
    return result, index

# -----
# Helper: decode 64-bit (double)
# -----
import struct
def read_64bit(buffer, index):
    data = buffer[index:index+8]
```

```

    value = struct.unpack("<d", data)[0] # little-endian double
    return value, index + 8

# -----
# Helper: recursively decode a nested message
# -----
def parse_message(buffer, index, end):
    fields = {}
    while index < end:
        # Decode tag
        tag, index = read_varint(buffer, index)
        field_number = tag >> 3
        wire_type = tag & 0x07

        # Decode by wire type
        if wire_type == WIRE_VARINT:
            value, index = read_varint(buffer, index)

        elif wire_type == WIRE_64BIT:
            value, index = read_64bit(buffer, index)

        elif wire_type == WIRE_LENGTH_DELIMITED:
            length, index = read_varint(buffer, index)
            value_bytes = buffer[index:index+length]
            index += length

            # Try nested message parse
            # For Timestamp and strings
            try:
                # Attempt nested decode
                nested = parse_message(value_bytes, 0, len(value_bytes))
                value = nested
            except Exception:
                # Otherwise it's string
                value = value_bytes.decode("utf-8")

        else:
            raise Exception(f"Unsupported wire type: {wire_type}")

        # Store field
        fields.setdefault(field_number, []).append(value)

    return fields

# -----
# Manual ParseFromString
# -----
def manual_parse(hex_data):
    raw = binascii.unhexlify(hex_data)
    print(f"Input bytes length: {len(raw)}")

```

```

    decoded = parse_message(raw, 0, len(raw))
    return decoded

# -----
# Test with your data
# -----
if __name__ == "__main__":
    hex_data = (
        "0801120e5355535465636820486f6f6469651a63f09fa7a5204120636f7a792c2073"
        "74796c69736820686f6f64696520666561747572696e6720746865206f6666696369"
        "616c2053555354656368206c6f676f2c207065726665637420666f722073686f7769"
        "6e67207363686f6f6c207370697269742e22074170706172656c291f85eb51b8fe48"
        "40321653746179207761726d2c20737461792070726f756421420c08d4f39dc80610"
        "a0a6d69f03"
    )

    msg = manual_parse(hex_data)

    print("\nDecoded fields:")
    for field, values in msg.items():
        print(f"Field {field}:")
        for v in values:
            print("    ", v)

```

Then we can get the result:

```

(base) cc@taskflow01:~/Distributed-and-Cloud-Computing/assignment2/codebase/src/api_service/server/python-flask$ pyth
hon decode.py
Input bytes length: 175

Decoded fields:
Field 1:
1
Field 2:
SUSTech Hoodie
Field 3:
🧡 A cozy, stylish hoodie featuring the official SUSTech logo, perfect for showing school spirit.
Field 4:
Apparel
Field 5:
49.99
Field 6:
Stay warm, stay proud!
Field 8:
{1: [1762097620], 2: [871732000]}
(base) cc@taskflow01:~/Distributed-and-Cloud-Computing/assignment2/codebase/src/api_service/server/python-flask$

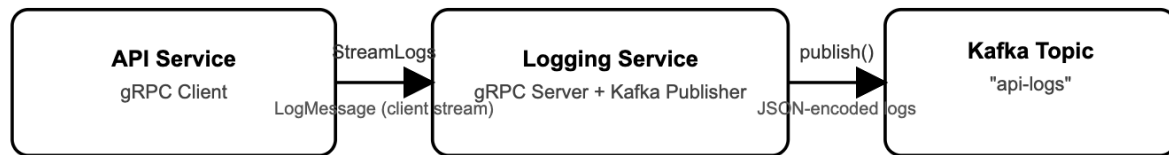
```

7. client-side streaming gRPC

This section we will introduce and explain how the client-side streaming RPC works in our Logging Service.

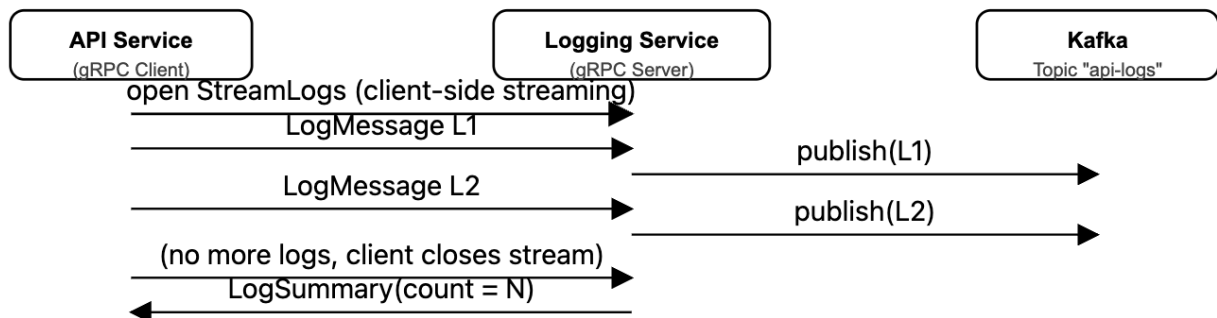
In our implementation, the Logging Service defines a **client-side streaming RPC** named `StreamLogs`. The RESTful API Service acts as the gRPC client and continuously sends log messages through a single streaming RPC call, while the Logging Service receives these messages and forwards them to Kafka.

Client-side Streaming Logging Pipeline



During runtime, API Service will send StreamLogs to Logging Service, and it will publish them to Kafka. Then if no more logs, client closes stream. So by doing that we don't need to have every LogMessage with every connections since ,any log messages share a single gRPC connection.

Sequence of Client-side Streaming Logs



1. Client Implementation (API Service)

On the client side, we implement a helper function `send_logs(logs)` :

```
def send_logs(logs):
    stub = get_logging_stub()
    INSTANCE_NAME = os.getenv("INSTANCE_NAME", "unknown")

    def gen():
        for log in logs:
            yield logging_pb2.LogMessage(
                service_name=INSTANCE_NAME,
                level=log.get("level", "INFO"),
                path=log.get("path", ""),
                method=log.get("method", ""),
                user_sid=log.get("user_sid", ""),
                message=log.get("message", ""),
                timestamp_ms=int(time.time() * 1000)
            )

    return stub.StreamLogs(gen())
```

Here:

- `get_logging_stub()` creates (and reuses) a gRPC channel and a `LoggingServiceStub` bound to `LOGGING_ADDR`.
- `send_logs(logs)` takes a list of Python dictionaries representing log entries.
- Inside `send_logs`, we define a generator function `gen()` that:
 - Iterates over the input list `logs`.
 - Converts each dictionary into a `LogMessage` protobuf.
 - Uses `yield` to stream messages one by one.

We then call `stub.StreamLogs(gen())`. This passes the generator `gen()` as the `request_iterator` for the `StreamLogs` RPC. The RPC is **client-side streaming**: multiple `LogMessage` instances are sent over a single RPC call, and the server returns a `LogSummary` only after the stream is closed.

2. Server Implementation (Logging Service)

On the server side, we implement the `LoggingService` as follows:

```
class LoggingService(logging_pb2_grpc.LoggingServiceServicer):
    def __init__(self, publisher):
        self.publisher = publisher

    def StreamLogs(self, request_iterator, context):
        count = 0
        for req in request_iterator:
            payload = {
                "service_name": req.service_name,
                "level": req.level,
                "path": req.path,
                "method": req.method,
                "user_sid": req.user_sid,
                "message": req.message,
                "timestamp_ms": req.timestamp_ms
            }
            try:
                self.publisher.publish(
                    topic=KAFKA_TOPIC,
                    key=req.service_name.encode(),
                    value=json.dumps(payload).encode()
                )
                count += 1
            except Exception as e:
                LOGGER.error("Publish fail: %s", e)

        LOGGER.info("Received %d logs", count)
        return logging_pb2.LogSummary(count=count)
```

Key points:

- `request_iterator` is the server-side view of the client stream. Each time the client's generator `gen()` yields a `LogMessage`, the `for req in request_iterator` loop receives it.
- For each `LogMessage` :
 - We convert it into a JSON payload.
 - We call `self.publisher.publish(...)` to send it to the Kafka topic `api-logs`, using the service name as the message key.
 - We increment `count` to track how many log entries were received in this stream.
- After the client finishes sending logs and closes the stream:
 - The loop terminates.
 - The server logs the total number of messages.
 - The server returns a `LogSummary(count=count)` back to the client, completing the RPC.

Summary of the Streaming Behavior

- **Single RPC, multiple messages:** The API Service opens one `StreamLogs` RPC and streams multiple `LogMessage` objects through it.
- **Streaming generator:** On the client, a Python generator function (`gen()`) produces the log messages lazily for the gRPC stream.
- **Incremental processing:** On the server, `StreamLogs` processes each incoming message immediately and publishes it to Kafka.
- **Final aggregation:** After all log messages are sent, the server replies with a single `LogSummary`, indicating how many logs were processed for this stream.

This design leverages gRPC client-side streaming to build an efficient, low-overhead logging pipeline from the API Service to Kafka via the Logging Service.

7. Docker & Docker Compose

In my implementation, I use a single Docker Compose setup so that all containers automatically join the same user-defined bridge network and can resolve each other by service name. PostgreSQL, Kafka, the DB Service, the Logging Service, and the RESTful API Service are all declared as services in `docker-compose.yml`.

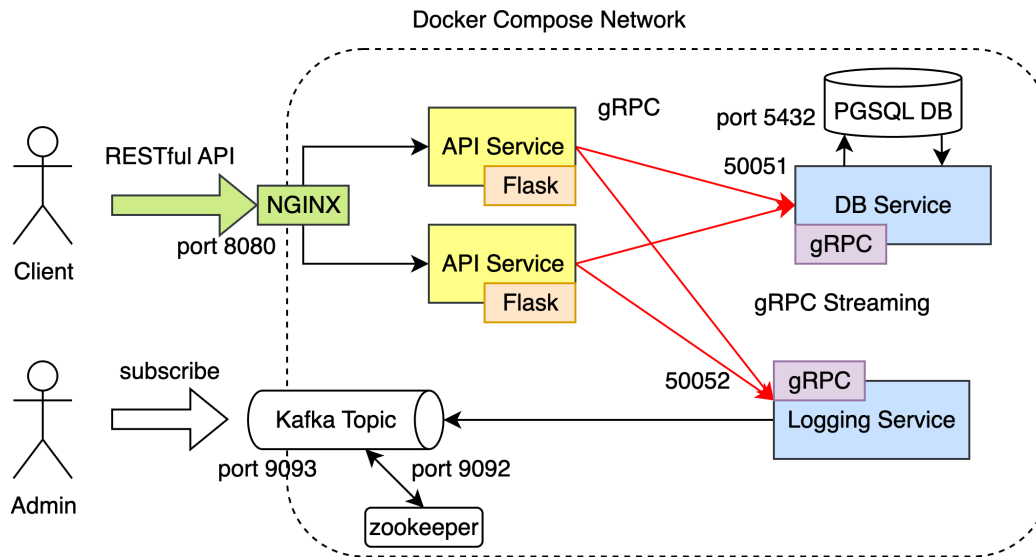
Single Docker Compose network = shared “LAN”

- Each service is reachable by its **service name** as a hostname
 - `db_service` → gRPC DB server
 - `logging_service` → gRPC logging server
 - `kafka` → Kafka broker
 - `postgres` → PostgreSQL
 - `api1`, `api2` → RESTful API instances
 - `NGINX` → `api1`, `api2`
- No need to hard-code IPs; Docker provides **internal DNS**.

Configure DB Service ↔ PostgreSQL: port 5432

Configure API Service ↔ DB Service & Logging Service: port 50051 & 50052

Configure Logging Service ↔ Kafka: port 9092



Inside the network, I use service names as hostnames (e.g., postgres:5432, db_service:50051, logging_service:50052, kafka:9092). I pass these endpoints into the containers via environment variables such as DB_HOST=postgres, DB_ADDR=db_service:50051, and KAFKA_BOOTSTRAP_SERVERS=kafka:9092.

The API Service connects to the DB Service and Logging Service via gRPC over these internal addresses, and the DB/Logging services connect to PostgreSQL and Kafka in the same way. For external access, I only expose the necessary ports (e.g., NGINX at localhost:8080 and Kafka at localhost:9092) to the host.

```
services:
  # database
  postgres:
    image: postgres:17
    container_name: postgres
    hostname: postgres
    environment:
      POSTGRES_USER: dncc
      POSTGRES_PASSWORD: dncc
      POSTGRES_DB: goodsstore
    ports:
      - "5432:5432" # Expose PostgreSQL to "local:container"
    volumes:
      - pg_data:/var/lib/postgresql/data
      - ./db-init:/docker-entrypoint-initdb.d

  # kafka message queue with zookeeper config management
  zookeeper:
    same logic as codebase provided
    ...

  kafka:
    same logic as codebase provided
```

```

...
...
ports:
  - "9092:9092" # For internal Docker use
  - "9093:9093" # For external access
...

kafka-topic-creator:
  image: confluentinc/cp-kafka:7.7.1
  depends_on:
    - kafka
  # This helper should not restart automatically after it finishes
  restart: "no"
  entrypoint: >
    bash -c "
      /bin/kafka-topics --create --topic log-channel --bootstrap-server kafka:9092 --
partitions 1 --replication-factor 1;
    "

```

For our implementation

```

##### YOUR SERVICES GO BELOW THIS LINE #####

db_service:
  build: ./src/db_service      # 根据你的真实路径改
  container_name: db_service
  depends_on:
    - postgres
  environment:
    POSTGRES_DSN: "dbname=goodsstore user=dncc password=dncc host=postgres
port=5432"

# ----- API 副本 -----
api1:
  build:
    context: ./src/api_service/server/python-flask/
  container_name: api1
  env_file:
    - .env
  environment:
    - LOGGING_ADDR=logging_service:50052
    - INSTANCE_NAME=api1
    - DB_ADDR=db_service:50051
  ports: []
  depends_on:
    - db_service
    - logging_service

api2:
  build:
    context: ./src/api_service/server/python-flask/

```

```

    container_name: api2
    env_file:
      - .env
    environment:
      - INSTANCE_NAME=api2
      - LOGGING_ADDR=logging_service:50052
      - DB_ADDR=db_service:50051
    ports: []
    depends_on:
      - db_service
      - logging_service

# ----- NGINX -----
nginx:
  image: nginx:alpine
  container_name: nginx_lb
  depends_on:
    - api1
    - api2
  ports:
    - "8080:80"          # external access http://localhost:8080 -> NGINX
  volumes:
    - ./nginx.conf:/etc/nginx/nginx.conf:ro
  networks:
    - default

logging_service:
  build:
    context: ./src/logging_service
    dockerfile: Dockerfile
  container_name: logging_service
  hostname: logging_service
  depends_on:
    - kafka
  environment:
    - KAFKA_BOOTSTRAP_SERVERS=kafka:9092
    - KAFKA_TOPIC=api-logs
  # internal gRPC
  ports:
    - "50052:50052"
  networks:
    - default

volumes:
  pg_data: # A placeholder volume without any configuration

```

NGINX:

```

services:
  nginx:

```

```
image: nginx:latest
container_name: nginx
ports:
  - "8080:80"    # host:8080 -> nginx:80
volumes:
  - ./nginx.conf:/etc/nginx/nginx.conf:ro
depends_on:
  - api1
  - api2
```

And inside `nginx.conf`:

```
http {
    upstream api_backend {
        server api1:8080;
        server api2:8080;
    }

    server {
        listen 80;
        location / {
            proxy_pass http://api_backend;
        }
    }
}
```

NGINX reaches the API instances using `api1` and `api2` service names, again via the same Docker network.

From outside, I only expose NGINX:

User → `http://localhost:8080`

NGINX → `api1:8080`, `api2:8080` inside Docker

This configuration allows all microservices to communicate reliably inside the Docker network while keeping internal ports isolated from the outside.

8. Experiments

Swagger UI / Kafka Experiments + Simple QPS Benchmark

Swagger UI

I mainly use Swagger UI to test my API Service.

First, User can see our website and its products:

"/"

Curl

```
curl -X 'GET' \
  'http://localhost:8080/' \
  -H 'accept: text/plain'
```

Request URL

http://localhost:8080/

Server response

Code

Details

200

Response body

Hello SUSTech Goods Store!

Download

Response headers

connection: keep-alive
content-length: 26
content-type: text/plain; charset=utf-8
date: Wed, 19 Nov 2025 16:51:05 GMT
server: nginx/1.29.3
x-upstream-addr: 172.18.0.0:8080

"/products"

Curl

```
curl -X 'GET' \
  'http://localhost:8080/products' \
  -H 'accept: application/json'
```

Request URL

http://localhost:8080/products

Server response

Code

Details

200

Response body

```
{
  "code": 200,
  "data": {
    "items": [
      {
        "category": "Apparel",
        "created_at": "2025-11-02T15:33:40.871732+00:00",
        "description": "🧢 A cozy, stylish hoodie featuring the official SUSTech logo, perfect for showing school spirit.",
        "id": 1,
        "name": "SUSTech Hoodie",
        "price": 49.99,
        "slogan": "Stay warm, stay proud!",
        "stock": 0
      },
      {
        "category": "Drinkware",
        "created_at": "2025-11-02T15:33:40.871732+00:00",
        "description": "💧 A high-quality, eco-friendly stainless steel water bottle with SUSTech branding.",
        "id": 2,
        "name": "SUSTech Water Bottle",
        "price": 19.98,
        "slogan": "Hydrate with pride!",
        "stock": 500
      }
    ]
  }
}
```

Download

Response headers

connection: keep-alive
content-length: 1299
content-type: application/json
date: Wed, 19 Nov 2025 16:52:02 GMT
server: nginx/1.29.3
x-upstream-addr: 172.18.0.9:8080

"/products/id"

Name

Description

id required

integer
(path)

1

Execute

Clear

Responses

Curl

```
curl -X 'GET' \
  'http://localhost:8080/products/1' \
  -H 'accept: application/json'
```

Request URL

http://localhost:8080/products/1

Server response

Code

Details

200

Response body

```
{
  "code": 200,
  "data": {
    "category": "Apparel",
    "created_at": "2025-11-02T15:33:40.871732+00:00",
    "description": "🧢 A cozy, stylish hoodie featuring the official SUSTech logo, perfect for showing school spirit.",
    "id": 1,
    "name": "SUSTech Hoodie",
    "price": 49.99,
    "slogan": "Stay warm, stay proud!",
    "stock": 0
  },
  "message": "Product found"
}
```

Download

Response headers

connection: keep-alive
content-length: 394
content-type: application/json
date: Wed, 19 Nov 2025 16:52:40 GMT
server: nginx/1.29.3
x-upstream-addr: 172.18.0.8:8080

Then, user can register:

```
{
  "email": "12211612@mail.sustech.edu.cn",
  "password": "123",
  "sid": "1612",
  "username": "BIN"
}
```

Execute

Clear

Responses

Curl

```
curl -X 'POST' \
  'http://localhost:8080/users/register' \
  -H 'accept: application/json' \
  -H 'Content-Type: application/json' \
  -d '{
    "email": "12211612@mail.sustech.edu.cn",
    "password": "123",
    "sid": "1612",
    "username": "BIN"
  }'
```

Request URL

```
http://localhost:8080/users/register
```

Server response

Code	Details
------	---------

201

Response body

```
{
  "code": 201,
  "data": {
    "token": "eyJ3bGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJzaWQ0IjoiXmJyIiwidXNlciI9pZCIG6JmJsImIhdCI6MTc2MzZuU3MTI1OCwiZXhwIjoxNzYzNjE0NDU0fQ.0r6tQQNjJlQw4VgI_bo7a2wuukURxa5fN7ZXqKScbyU",
    "user": {
      "created_at": "2025-11-19T16:54:18.203499+00:00",
      "email": "12211612@mail.sustech.edu.cn",
      "is_active": true,
      "sid": "1612",
      "username": "8IN"
    }
  },
  "message": "User registered successfully"
}
```

Response headers

```
connection: keep-alive
content-length: 453
content-type: application/json
date: Wed, 19 Nov 2025 16:54:18 GMT
server: nginx/1.29.3
x-upstream-addr: 172.18.0.8:8080
```

User can login and get a JWT. And we won't get passwd back.

Request body

Required

application/json

{
 "password": "123",
 "said": "1612"
}

+🔍

Execute

Clear

Responses

Curl

```
curl -X 'POST' \  
  'http://localhost:8080/users/login' \  
  -H 'accept: application/json' \  
  -H 'Content-Type: application/json' \  
  -d '{  
    "password": "123",  
    "said": "1612"  
  }'
```

📄

Request URL

```
http://localhost:8080/users/login
```

Server response

Code

Details

200

Response body

```
{  
  "code": 200,  
  "data": {  
    "token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJzdWIjOiIxNjEwMmQzMzYyZnVlbnQNTeYfQ.FX6V4keZegiRIbb6W9BiJmlUYzLyLQy4gQioguNsNwf  
g",  
    "user": {  
      "created_at": "2025-11-19T16:54:18.203499+00:00",  
      "email": "12211612@mail.sustech.edu.cn",  
      "is_active": true,  
      "sid": "1612",  
      "username": "BIN"  
    }  
  },  
  "message": "Login successful"  
}
```

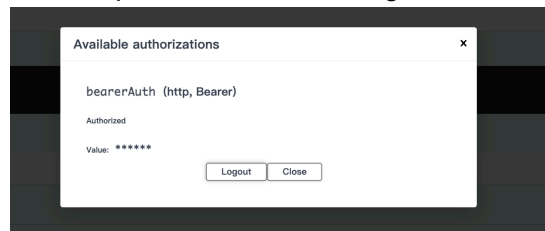
📄

Download

Response headers

```
connection: keep-alive  
content-length: 441  
content-type: application/json  
date: Wed, 19 Nov 2025 16:55:12 GMT  
server: nginx/1.29.3  
x-upstream-addr: 172.18.0.9:8080
```

We can put the JWT as we login:



Get orders:

[illegible]

Place Order

```
{
  "product_id": 2,
  "quantity": 2
}
```

[illegible]

The order id is 256.

We can delete it:

DELETE

/orders/{id} Cancel an order

Parameters

Name	Description
id * required	
integer	256
(path)	

Execute

Clear

Responses

Curl

```
curl -X 'DELETE' \
'http://localhost:8080/orders/256' \
-H 'accept: */*' \
-H 'Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJzaWQiOiIxNjEyIiwidXNlci9pZCI6ImJmInhdCI6MTc2MzU3MTU2LWlZXIwIjoxNzYzNjE0NzY2RQ. nr9EBNgZZnJdT-_jNvNnI1VZeulGRxm9buffZ5jo818'
```

Request URL

http://localhost:8080/orders/256

Server response

Code	Details
200	Response body <pre>{ "code": 200, "data": null, "message": "Order deleted" }</pre> Download

Response headers

```
connection: keep-alive
content-length: 64
content-type: application/json
date: Wed, 18 Nov 2025 17:18:14 GMT
server: nginx/1.29.3
x-upstream-addr: 172.18.0.8:8080
```

Responses

Code	Description	Links
------	-------------	-------

Then:

GET /orders/{id} Get an order by id (current user's order)

Parameters

Cancel

Name	Description
id * required integer (path)	256

Execute Clear

Responses

Curl

```
curl -X 'GET' \
  'http://localhost:8080/orders/256' \
  -H 'accept: application/json' \
  -H 'Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJzdWIjOiI1aXNjYiIiwidmVlcnQ9ZCJGMjMzU3MTU2NiwiZXN1Ijo4NzYzJiE9NzYzFQ.rr9EBNgZZnJdT_JhVWn1IvIeulaBom9urFTZ5j081t'
```

Request URL

http://localhost:8080/orders/256

Server response

Code Details

404

Error: NOT FOUND

Response body

```
{
  "code": 404,
  "data": null,
  "message": "Order not found"
}
```

Response headers

```
connection: keep-alive
content-length: 66
content-type: application/json
date: Wed, 19 Nov 2025 17:15:04 GMT
server: nginx/1.29.3
```

Responses

Check my account status:

GET /user/me

Curl

```
curl -X 'GET' \
  'http://localhost:8080/users/me' \
  -H 'accept: application/json' \
  -H 'Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJzdWIjOiI1aXNjYiIiwidmVlcnQ9ZCJGMjMzU3MTU2NiwiZXN1Ijo4NzYzJiE9NzYzFQ.rr9EBNgZZnJdT_JhVWn1IvIeulaBom9urFTZ5j081t'
```

Request URL

http://localhost:8080/users/me

Server response

Code Details

200

Response body

```
{
  "code": 200,
  "data": {
    "created_at": "2025-11-19T16:54:18.283499+00:00",
    "email": "12211612@mail.sustech.edu.cn",
    "is_active": true,
    "id": "1612",
    "username": "BIN"
  },
  "message": "User profile fetched"
}
```

Response headers

```
connection: keep-alive
content-length: 235
content-type: application/json
date: Wed, 19 Nov 2025 17:28:58 GMT
server: nginx/1.29.3
x-upstream-addr: 172.18.0.1:8080
```

Responses

Update my account:

PATCH /user/me


```
docker exec -it kafka kafka-console-consumer --bootstrap-server localhost:9092
--topic api-logs --from-beginning
```

we can see the log from admin side.

```
2025-11-16 17:04:32,663 WARN [Consumer clientId=console-consumer, groupId=console-consumer-80274] Connection to node 1 (kafka/172.18.0.5:9092) could not be established. Node
may not be available. (org.apache.kafka.clients.NetworkClient)
2025-11-16 17:04:32,900 WARN [Consumer clientId=console-consumer, groupId=console-consumer-80274] Connection to node 1 (kafka/172.18.0.5:9092) could not be established. Node
may not be available. (org.apache.kafka.clients.NetworkClient)
(base) cc@taskflow01:~/Distributed-and-Cloud-Computing/assignment2/codebase$ docker exec -it kafka kafka-console-consumer --bootstrap-server localhost:9092 --topic api-
logs --from-beginning
[2025-11-16 17:04:42,289] WARN [Consumer clientId=console-consumer, groupId=console-consumer-20677] Error while fetching metadata with correlation id 2 : (api-logs=LEADER_NOT_A
VAILABLE) (org.apache.kafka.clients.NetworkClient)
{"service_name": "api1", "level": "INFO", "path": "/products", "method": "GET", "user_sid": "", "message": "Products fetched: 3", "timestamp_ms": 1763312687848}
{"service_name": "api2", "level": "INFO", "path": "/products/1", "method": "GET", "user_sid": "", "message": "Product 1 fetched", "timestamp_ms": 1763312696085}
{"service_name": "api1", "level": "INFO", "path": "/products/1", "method": "GET", "user_sid": "", "message": "Product 1 fetched", "timestamp_ms": 1763312696835}
{"service_name": "api2", "level": "INFO", "path": "/products/1", "method": "GET", "user_sid": "", "message": "Product 1 fetched", "timestamp_ms": 1763312697637}
{"service_name": "api1", "level": "INFO", "path": "/", "method": "GET", "user_sid": "", "message": "Root endpoint accessed", "timestamp_ms": 176331867854}
{"service_name": "api2", "level": "INFO", "path": "/", "method": "GET", "user_sid": "123", "message": "User registered successfully", "timestamp_ms": 1763319468412}
{"service_name": "api1", "level": "INFO", "path": "/users/login", "method": "POST", "user_sid": "123", "message": "Login successful", "timestamp_ms": 1763319485421}
{"service_name": "api2", "level": "INFO", "path": "/users/me", "method": "GET", "user_sid": "123", "message": "User profile fetched successfully", "timestamp_ms": 1763319525942}
{"service_name": "api1", "level": "INFO", "path": "/orders", "method": "GET", "user_sid": "", "message": "Orders fetched successfully", "timestamp_ms": 1763319692571}
{"service_name": "api2", "level": "INFO", "path": "/orders", "method": "POST", "user_sid": "", "message": "Order created successfully", "timestamp_ms": 1763319710084}
{"service_name": "api1", "level": "INFO", "path": "/orders", "method": "POST", "user_sid": "", "message": "Order created successfully", "timestamp_ms": 1763319816423}
{"service_name": "api1", "level": "INFO", "path": "/", "method": "GET", "user_sid": "", "message": "Root endpoint accessed", "timestamp_ms": 1763320618899}
```

QPS Benchmark

we use hey to benchmark our APIs.

```
sudo apt install hey
# Run the testing
hey -n 2000 -c 50 http://localhost:8080/products
```

2000 means we request 2000 times, while 50 means 50 queries at one time.

There will be several info we can get to see if our API is fast enough, and what's the bottleneck:

```
No VM guests are running outdated hypervisor (qemu) binaries on this host.
(cc2) cc@taskflow01:~/Distributed-and-Cloud-Computing/assignment2/codebase$ hey -n 2000 -c 50 http://localhost:8080/products

Summary:
Total:      2.2360 secs
Slowest:    0.1246 secs
Fastest:    0.0040 secs
Average:    0.0541 secs
Requests/sec: 894.4532

Total data: 2602000 bytes
Size/request: 1301 bytes

Response time histogram:
0.004 [1]
0.016 [17]
0.028 [27]
0.040 [237]
0.052 [701]
0.064 [580]
0.076 [300]
0.088 [102]
0.100 [26]
0.113 [7]
0.125 [2]

Latency distribution:
10% in 0.0382 secs
25% in 0.0440 secs
50% in 0.0525 secs
75% in 0.0628 secs
90% in 0.0731 secs
95% in 0.0790 secs
99% in 0.0957 secs

Details (average, fastest, slowest):
DNS+diialup: 0.0001 secs, 0.0040 secs, 0.1246 secs
DNS-lookup: 0.0000 secs, 0.0000 secs, 0.0063 secs
req write: 0.0000 secs, 0.0000 secs, 0.0018 secs
resp wait: 0.0539 secs, 0.0040 secs, 0.1181 secs
resp read: 0.0000 secs, 0.0000 secs, 0.0011 secs

Status code distribution:
[200] 2000 responses
```

As we can see, the QPS of our API is 894. According to the chart, most of the response time lies in 0.088s. And 95% in 0.0790s. The bottleneck is the DNS dialup (from docker) and resp wait (database service

query).

So, a good way to improve the latency may be using some of the memory database like `redis`. Redis can store and return the query result in memory instead of the SSD/HDD.

This shows the reason why so many companies try to use Redis.

We also test the api of `/orders`, with Auth. As we can see, this time `resp wait` increase, since we have more complex transaction logic.

```
(dcc2) cc@taskflow1:~/Distributed-and-Cloud-Computing/assignment2/codebase$ TOKEN="eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJ3aWQ1OiJxMjM1LCJ1c2VyX2lkIjoxOjUwS2VhZmZlE5NDg1LlCjleHA1OjE3NjMzNjI2ODV9.8akvMEJvNrV9zRR924eboTc9pF2x_5SM998Js2RgUIs"
hey -n 2000 -c 50 \
  -m POST \
  -H "Authorization: Bearer $TOKEN" \
  -H "Content-Type: application/json" \
  -H "accept: application/json" \
  -d '{"product_id":1,"quantity":2}' \
  http://localhost:8080/orders

Summary:
  Total:      2.4961 secs
  Slowest:    0.1735 secs
  Fastest:    0.0090 secs
  Average:    0.0608 secs
  Requests/sec: 801.2629

  Total data: 242469 bytes
  Size/request: 121 bytes

Response time histogram:
  0.009 [1] |
  0.025 [30] |
  0.042 [161] |
  0.058 [753] |
  0.075 [730] |
  0.091 [226] |
  0.108 [60] |
  0.124 [26] |
  0.141 [11] |
  0.157 [1] |
  0.174 [1] |

Latency distribution:
  10% in 0.0421 secs
  25% in 0.0480 secs
  50% in 0.0593 secs
  75% in 0.0700 secs
  90% in 0.0810 secs
  95% in 0.0910 secs
  99% in 0.1191 secs

Details (average, fastest, slowest):
  DNS+ Dialup: 0.0001 secs, 0.0000 secs, 0.1735 secs
  DNS-lookup: 0.0000 secs, 0.0000 secs, 0.0078 secs
  req write: 0.0000 secs, 0.0000 secs, 0.0062 secs
  resp wait: 0.0606 secs, 0.0089 secs, 0.1735 secs
  resp read: 0.0000 secs, 0.0000 secs, 0.0012 secs

Status code distribution:
  [201] 242 responses
```

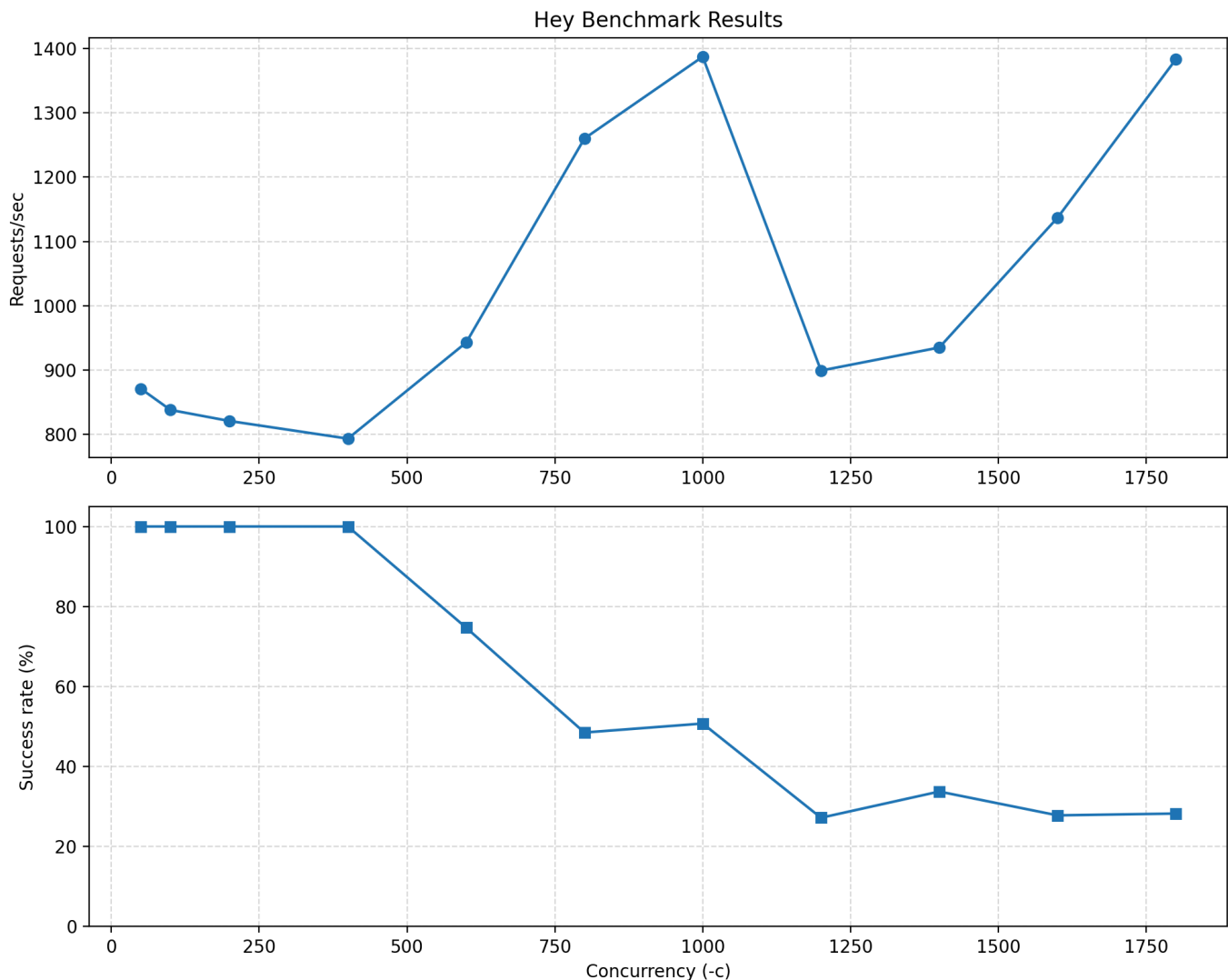
(tips: since we only have 484 products (Originally 500, some of them I ordered before), only 242 requests success. But this maybe a good case for **double-eleven day** “秒杀” or why our school choose class systems fail all the time)

Understanding NGINX

Here I do a very simple experiment on Request. As we increased the concurrency from 50 to 400, the system was able to handle all 2000 requests successfully, and the throughput (Requests/sec) gradually increased.

Beyond a concurrency of around 600, the system started dropping requests: many connections failed at the transport layer and never produced an HTTP status code, eventually falling below 30%, while the apparent

Requests/sec curve peaks around 1000 concurrency, drops, and then unexpectedly rises again.



As you can see, when concurrency is 400 (that means 400 users request at one time), the success rate is 100% and dealing speed is about 800. This maybe the maximum number that system can take. After that, system can not deal with some request.

Why the Success rate go down as concurrency increase

When concurrency becomes too high for the system to handle, many requests begin failing **before they even reach the application layer**. This happens due to limits in the web server (NGINX), OS file descriptors, or the backend worker capacity.

Once these limits are exceeded:

- New TCP connections are **rejected, reset, or timeout**
- These failed requests **never receive an HTTP status code**
- **Only a subset of requests successfully passes through NGINX and reaches Flask**
- Therefore, the number of successful HTTP 200 responses decreases

So the success rate decreases not because the API behaves incorrectly, but because **the system is overloaded and cannot accept all incoming connections**, causing many requests to fail at the transport

layer.

Why the Request speed reach limit at 1000 then drop, then rise up:

This is mainly because of NGINX.

This behavior is a typical artifact caused by **connection limits inside NGINX** and how **hey** calculates throughput.

1) NGINX has a connection limit (~1024 worker_connections)

Once concurrency approaches or exceeds this limit:

- Only a fixed number of connections are allowed to be active
- Extra connections fail immediately without entering the queue

2) Failed connections return instantly, reducing total test duration

hey computes Requests/sec as: `successful_requests / total_test_time`

When concurrency is extremely high:

- Many requests fail instantly (connection refused/reset)
- The total test duration becomes **very short**
- The small number of successful requests complete quickly
- Therefore, `successful_requests / time` may appear **higher again**, even though the system is overloaded

3) This creates the illusion that the throughput rises again

In reality:

- The backend is *not* performing better
- The increase is an artifact caused by:
 - NGINX rejecting excess connections immediately
 - hey shortening the run duration
 - Only the “lucky” in-limit connections being measured

So the rise after 1000 concurrency is not true performance gain, but **a side effect of NGINX’s connection limit and hey’s measurement method**.

Therefore:

The Requests/sec curve peaks, drops, and rises again mainly because of NGINX’s worker connection limits and how hey measures throughput.

Some we learn innerly: The Power of NGINX

This behavior actually highlights one of the advantages of using NGINX.

When the concurrency exceeds the server’s capacity, NGINX intentionally rejects excessive connections

immediately. This “fail-fast” behavior prevents resource exhaustion, protects backend services, and ensures that the valid in-limit connections can still be processed quickly.

In other words, instead of letting the entire system become slow or crash, NGINX performs graceful degradation:

it sacrifices excess traffic to preserve the performance and stability of the remaining requests.

Kafka now is very crazy:

```
p_ms": 1763321281301}
{"service_name": "api2", "level": "ERROR", "path": "/orders", "method": "POST", "user_sid": "", "message": "DB service error: FAILED_PRECONDITION - not enough stock", "timestamp": 1763321281301}
{"service_name": "api2", "level": "ERROR", "path": "/orders", "method": "POST", "user_sid": "", "message": "DB service error: FAILED_PRECONDITION - not enough stock", "timestamp": 1763321281303}
{"service_name": "api1", "level": "ERROR", "path": "/orders", "method": "POST", "user_sid": "", "message": "DB service error: FAILED_PRECONDITION - not enough stock", "timestamp": 1763321281303}
{"service_name": "api1", "level": "ERROR", "path": "/orders", "method": "POST", "user_sid": "", "message": "DB service error: FAILED_PRECONDITION - not enough stock", "timestamp": 1763321281304}
{"service_name": "api2", "level": "ERROR", "path": "/orders", "method": "POST", "user_sid": "", "message": "DB service error: FAILED_PRECONDITION - not enough stock", "timestamp": 1763321281305}
{"service_name": "api1", "level": "ERROR", "path": "/orders", "method": "POST", "user_sid": "", "message": "DB service error: FAILED_PRECONDITION - not enough stock", "timestamp": 1763321281310}
{"service_name": "api1", "level": "ERROR", "path": "/orders", "method": "POST", "user_sid": "", "message": "DB service error: FAILED_PRECONDITION - not enough stock", "timestamp": 1763321281311}
{"service_name": "api2", "level": "ERROR", "path": "/orders", "method": "POST", "user_sid": "", "message": "DB service error: FAILED_PRECONDITION - not enough stock", "timestamp": 1763321281314}
{"service_name": "api1", "level": "ERROR", "path": "/orders", "method": "POST", "user_sid": "", "message": "DB service error: FAILED_PRECONDITION - not enough stock", "timestamp": 1763321281314}
{"service_name": "api2", "level": "ERROR", "path": "/orders", "method": "POST", "user_sid": "", "message": "DB service error: FAILED_PRECONDITION - not enough stock", "timestamp": 1763321281315}
{"service_name": "api2", "level": "ERROR", "path": "/orders", "method": "POST", "user_sid": "", "message": "DB service error: FAILED_PRECONDITION - not enough stock", "timestamp": 1763321281316}
{"service_name": "api2", "level": "ERROR", "path": "/orders", "method": "POST", "user_sid": "", "message": "DB service error: FAILED_PRECONDITION - not enough stock", "timestamp": 1763321281320}
{"service_name": "api1", "level": "ERROR", "path": "/orders", "method": "POST", "user_sid": "", "message": "DB service error: FAILED_PRECONDITION - not enough stock", "timestamp": 1763321281321}
{"service_name": "api2", "level": "ERROR", "path": "/orders", "method": "POST", "user_sid": "", "message": "DB service error: FAILED_PRECONDITION - not enough stock", "timestamp": 1763321281322}
{"service_name": "api1", "level": "ERROR", "path": "/orders", "method": "POST", "user_sid": "", "message": "DB service error: FAILED_PRECONDITION - not enough stock", "timestamp": 1763321281323}
{"service_name": "api1", "level": "ERROR", "path": "/orders", "method": "POST", "user_sid": "", "message": "DB service error: FAILED_PRECONDITION - not enough stock", "timestamp": 1763321281323}
{"service_name": "api2", "level": "ERROR", "path": "/orders", "method": "POST", "user_sid": "", "message": "DB service error: FAILED_PRECONDITION - not enough stock", "timestamp": 1763321281323}
{"service_name": "api2", "level": "ERROR", "path": "/orders", "method": "POST", "user_sid": "", "message": "DB service error: FAILED_PRECONDITION - not enough stock", "timestamp": 1763321281327}
{"service_name": "api1", "level": "ERROR", "path": "/orders", "method": "POST", "user_sid": "", "message": "DB service error: FAILED_PRECONDITION - not enough stock", "timestamp": 1763321281327}
{"service_name": "api2", "level": "ERROR", "path": "/orders", "method": "POST", "user_sid": "", "message": "DB service error: FAILED_PRECONDITION - not enough stock", "timestamp": 1763321281329}
{"service_name": "api2", "level": "ERROR", "path": "/orders", "method": "POST", "user_sid": "", "message": "DB service error: FAILED_PRECONDITION - not enough stock", "timestamp": 1763321281331}
{"service_name": "api1", "level": "ERROR", "path": "/orders", "method": "POST", "user_sid": "", "message": "DB service error: FAILED_PRECONDITION - not enough stock", "timestamp": 1763321281331}
{"service_name": "api1", "level": "ERROR", "path": "/orders", "method": "POST", "user_sid": "", "message": "DB service error: FAILED_PRECONDITION - not enough stock", "timestamp": 1763321281332}
{"service_name": "api2", "level": "ERROR", "path": "/orders", "method": "POST", "user_sid": "", "message": "DB service error: FAILED_PRECONDITION - not enough stock", "timestamp": 1763321281337}
{"service_name": "api1", "level": "ERROR", "path": "/orders", "method": "POST", "user_sid": "", "message": "DB service error: FAILED_PRECONDITION - not enough stock", "timestamp": 1763321281340}
^CProcessed a total of 6012 messages
(base) ccc@taskflow01:~/Distributed-and-Cloud-Computing/assignment2/codebases$
```

For most of the CRUD scenario, the QPS is above 800, which is ok for a school market backend. But to increase, we can increase the number of thread in DB threadpool, gRPC pool, and API Server.

More over, we should develop a distributed cloud system to support them with more QPS support (This is the purpose why we attend the class :0).

There are so many other benchmarks we can make, like NGINX scheduling strategy, thread number, etc. You can try it later.

9. Possible Improvement

There are still many things we didn't do:

1. Soft deleting: we didn't delete data from db. Instead we just update a key into 'false'.
2. Multi-node: we can publish different docker instance in different nodes. Probably a K8s is needed for better management.
3. Some of the strange bugs that may happen.
4. Experiments on different thread number in threadpool
5. NGINX strategy

6. Multi language that may differ the QPS

But I think a good software, is not only equal to a software that without bugs/improvements. Win 7 is a good software, Linux is a good software. The scalability, user friendly, safety enough, easy to develop, these features are also important features.

So although we have so much to do, still we have done so much. And that's a wonderful journey so far!